RESEARCH-ARTICLE

An Empirical Study on the Characteristics of Reusable Code Clones

Published: 23 January 2026
Accepted: 06 January 2026
Revised: 27 September 2025
Received: 18 September 2024

An Empirical Study on the Characteristics of Reusable Code Clones

CHUNLI YU, Queen's University, Canada
SHAYAN NOEI, Queen's University, Canada
HAOXIANG ZHANG, Queen's University, Canada
YING ZOU, Queen's University, Canada

Copy-and-paste of code is a common practice in software development, especially with the availability of myriad open-source projects. Reusing existing code fragments may accelerate the development process and ensure better quality of software if high-quality code fragments are reused. Therefore, code cloning might not be avoided even though it can increase software maintenance cost as bugs can be propagated inadvertently by code cloning and go beyond the boundary of a system. Generally speaking, reusable code clones are highly popular, have fewer bugs, and can stand the test of time thus exist in the system for a long term. Current code reuse studies primarily leverage API usage and method clone structures to provide coding recommendations. However, the existing approaches focus on the functional utility of code snippets without considering the quality aspects, such as fault resiliency, when analyzing code reuse. Without clone quality in mind, the virtues of code cloning are severely diminished. To help developers determine if the code clones are reusable or not, we leverage Machine Learning classifiers and Large Language Models (LLMs) to automatically identify reusable code clones from three perspectives: clone prevalence (i.e., the number of clone siblings), clone fault-resiliency (i.e., the percentage of non-buggy commits versus buggy commits), and clone longevity (i.e., the clone genealogy length). Our approach achieves a median AUC of 0.73, with an F1-score of 0.89, based on experiments conducted on 60 open-source projects, consisting of 30 Java and 30 C projects, which collectively encompass 538,598 commits. The results show that CountFollowers (i.e., number of people following the contributors that wrote the clones), SimilarityClonePaths (i.e., the Jaccard similarity coefficient among clone paths), and CountContributors (i.e., number of distinct contributors that access the clone group) provide the most explanatory power that contributes to correctly classifying reusable code clones. Hence, practitioners can utilize our classifiers and insights from our findings to make more reliable use of code clones and prioritize the use of high-quality clones in their clone management activities.

CCS Concepts: • **Software and its engineering → Maintaining software**; **Software evolution**;

Additional Key Words and Phrases: Code Clone Reuse, Machine Learning, Large Language Models, Clone Evolution, Software Quality

1 Introduction

Source code reuse is widely adopted in software engineering [7, 27, 60]. Instead of reinventing the wheel, developers tend to copy-and-paste code fragments during the software development process, which results in duplicated or similar code fragments dispersed in a software systems. Such similar code snippets are termed as code clones, and a set of similar code fragments (i.e., clone siblings) forms a clone group (hereafter, clones). It is reported that a significant fraction ranging from 7% to 23% of the code in software systems has been cloned [38, 99, 123].

Authors' Contact Information: Chunli Yu, chunli.yu@queensu.ca, Queen's University, Kingston, Canada; Shayan Noei, s.noei@queensu.ca, Queen's University, Kingston, Canada; Haoxiang Zhang, haoxianghz@gmail.com, Queen's University, Kingston, Canada; Ying Zou, ying.zou@queensu.ca, Queen's University, Kingston, Canada.

ACM 1557-7392/2026/1-ART
https://doi.org/10.1145/3793251

Representing frequently repeated common functionalities in software systems, code clones are candidates for reuse in a rapid development process. Code cloning can also introduce ethical and licensing concerns. For instance, the GNU General Public License (GPL) follows strict copyright principles on code reuse [132]. Admittedly, code cloning can incur unexpected quality issues and unnecessary maintenance costs [65]. For instance, if a code fragment is buggy and a programmer copies that piece of code fragment to several other places, without developers' awareness of the bugs lurking in it, then the bugs get replicated. In addition, bug fixing involving a single copy of clone fragments may need to be synchronized to all the other clone siblings, escalating the maintenance effort required for the developers. However, code cloning cannot be avoided due to its advantages in shortening the development cycle [28, 54]. By copying existing code fragments, code cloning can expedite the development process, thus boosting productivity [71]. Besides, it can be beneficial to ensure software quality if well-designed and tested code fragments are reused [72].

As part of the code cloning process, is to reuse existing code fragments, developers need to identify an asset (i.e., a block of code fragments, such as, a method or class) as functionally fitting for the desired task. It is desirable that code clones free of bugs are suggested as examples for future reuse. Current code reuse studies primarily leverage API usage [2, 5] and method clone structures [4, 45] to provide coding suggestions. However, these methodologies focus on the functional utility of code snippets and typically overlook bug proneness and long-term reusability. Ignoring these quality factors can increase the risk of introducing or propagating bugs and system failures through code cloning [78], and further deteriorate the code maintainability and reusability.

To assist developers in determining if the copied code can be reused reliably, we provide an approach that automatically classifies whether a cloned code snippet is reusable, considering the prevalence, fault resiliency, and longevity of the code [34]. As proposed in prior studies [37], the likelihood of code clones being reusable in terms of code functionality can be measured by clone occurrences. Reusable code clones should have higher quality, e.g., fewer bugs [36, 41]. Moreover, reusable code clones can stand the test of time thus exist in the system for a long term. Inspired by the reusable characteristics in the aforementioned studies, we attempt to create an appropriate reusable clone dataset - incorporating a series of thresholds (i.e., 0.3, 0.4, 0.5, 0.6, and 0.7) to define criteria (e.g., clone prevalence) for clones being reusable.

Methods or program functions can be considered as the simplest form of code reuse [47, 103]. Therefore, we investigate reusable code clones at the method level. To evaluate the performance of our classifiers, we conduct an empirical study on 60 open-source projects (i.e., 30 Java and 30 C projects) and frame our research around the following research questions (RQs):

RQ1: How well can we classify the reusable code clones? To reduce the manual effort in identifying reusable code clones, from a vast amount of cloned code (i.e., 7%-23% of code) in a software project, we use five Machine Learning classifiers and three popular open-source LLMs to automatically identify reusable code clones. The results of our clone classification model shows that the Random Forest algorithm can effectively identify reusable code clones, achieving an acceptable Area Under the ROC Curve (AUC) of 0.73 and an F1-score of 0.89. We further conduct sensitivity analysis to understand the effects of varying the thresholds of different selection criteria (e.g., clone prevalence) to create the dataset on the efficacy of machine learning classifiers in identifying reusable code clones. Our analysis reveals that a 0.5 threshold offers a balanced trade-off—yielding the highest Matthews Correlation Coefficient (MCC) of 0.31—while maintaining a manageable class imbalance of 12%.

RQ2: Which features in the classification model have the most explanatory power in distinguishing reusable code clones from non-reusable ones? Using 11 uncorrelated and non-redundant features, we explore the most influential features on the models and wish to understand the characteristics of reusable code clones. We find that *CountFollowers* (i.e., number of people following the contributors that wrote the clones.), *SimilarityClonePaths* (i.e., the Jaccard similarity coefficient among clone paths), and *CountContributors* (i.e., number of distinct contributors that access the clone group) are the most explanatory features to the models in identifying reusable code clones.

RQ3: What are the functionalities of reusable code clones? Among 11 general taxonomy of clone functionality, search and event-handling offer the greatest number of clones that can be reused. In cases where resources required to manage code clones exceed anticipated efforts, practitioners can prioritize on reusing code fragments associated with *search* and *event-handling* functionality.

Overall, our contributions are three-fold:

- We propose an approach that can automatically identify reusable code clones using machine learning classifiers. We achieve an AUC of 0.73 and an MCC of 0.31, with an F1-score of 0.89, using a threshold of 0.5 for classifying clones as reusable.
- We conduct a qualitative analysis of the predicted reusable clones and derive a category of reusable functionality in software engineering.
- We create a ground truth dataset of reusable code clones on the method-level granularity based on 60 large open-source Java (30) and C (30) projects, and evaluate our method for extracting reusable clones using the dataset.

The replication package of the study can be accessed online [128].

The rest of the paper is organized as follows. Section 2 explains the dataset collection and process overview. Section 3 presents the methods and results of our research questions. Section 4 discusses our implications. Section 5 provides threats to the validity of our study. Section 6 explores the related work relevant to our study. Finally, Section 7 summarizes this study and provides future directions.

2 Experiment Setup

In this section, we provide the experiment setup of our study.

2.1 Studied Projects

We focus our research on projects written in Java and C due to their popularity [121] and because these languages have historically been the primary focus of code cloning studies and benchmark tools [62, 113, 120, 124]. We identify active public repositories with the following criteria:

(i) The source code of the project has been modified at least once during the period from May 2022 to May 2023 to filter out inactive projects;

(ii) 10MB for the codebase's footprint size [26, 88] to filter out highly-starred but insubstantial projects, while ensuring the projects are large enough for meaningful code clone extraction. For instance, despite its high popularity (2.3K stars) and extensive activity (more than 700 issues and 300 PRs), the *helloworld* [1] GitHub project is largely a single README.md demo file and does not serve our research goals; *(iii)* At least ten contributors [92] to ensure the project has sufficient contributors for clone evolution, along with more than one year of project history, meaning the project has had active code commits over the past year. This criterion helps exclude toy or self-learning projects;

(iv) At least 500 commits [46, 70, 122], 500 closed issue reports, 500 pull requests, to ensure a long enough evolution history to study;

(v) We exclude forked repositories in this study. Forking a project means creating a copy of the original project, typically with the intention of independently developing it apart from the original project. This can cause a substantial code overlap between the original and forked projects, making it difficult to distinguish genuine instances of reusable code clones from those that are merely resulting from the forking process. In addition, we focus on the mainstream branch since many other branches are done simply to contribute a patch rather than instigating new development. Such branches may have substantial cloned code, but not necessarily due to code reuse intentions.

[1] https://github.com/octocat/hello-world

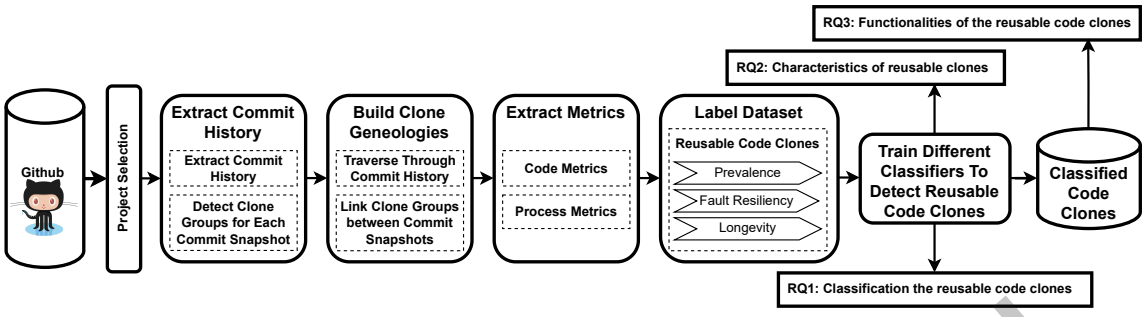

Fig. 1. The overview of the process used to conduct our experiment for identifying reusable clones.

We use the GitHub API [2] for identifying the projects. Due to the limitations of the GitHub API, we first collect projects with a history exceeding one year and with at least one commit occurring between May 2022 and May 2023, which results in 1,844 projects. We refined our list further by filtering out projects with fewer than 500 commits, 500 closed issues, or 500 PRs, ultimately narrowing our dataset to 515 projects.

Mining 515 repositories and creating a change history of all is not feasible due to the scalability of clone detection tools. We wish to study projects in different development characteristics, using commit activities as a criterion to differentiate projects. 515 projects are therefore categorized into three groups based on the number of commits of a project: those with an extended development history (i.e., the top 33%), those with a brief development history (i.e., the bottom 33%), and an intermediate category (i.e., the rest of the projects from 34%-67%). Employing a stratified random sampling strategy, we select 20 projects in each category — comprising 10 Java and 10 C projects — from each programming language, yielding 60 instances in total, to provide a balanced and representative dataset for our analysis. The descriptive statistics of studied projects reveals substantial diversity in the development activities, with commits between 1,841 and 50,380 and project sizes from 10.07 MB to 529.86 MB. Table 1 shows an overview of our studied projects.

2.2 Data Processing

Figure 1 outlines the steps for processing the data, including detecting clones, building clone group genealogies, calculating metrics to extract features for building classifiers, and processing the labeled dataset to create reusable code clones.

2.2.1 Detecting Clones. Since we detect clones for each snapshot of the project (*i.e.,* commit), we select token-based or text-based clone detection tools. Abstract syntax tree (AST) and graph-based clone detection tools, such as Deckard [51], require extracting syntax trees or relation graphs [125], which could be a time-consuming process to detect clones of a large-sized project. For instance, on larger projects, Deckard may take more than a day to generate a single snapshot. Since our analysis detects code clones for each commit, with some projects exceeding 20k commits/snapshots in order to build clone genealogies, we chose faster alternatives: token and text-based approaches, including SourcererCC [102], NiCad [98], NIL [81], and LvMapper [126]. LvMapper lacks open-source availability, and SourcererCC has not been updated since 2016. To the best of our knowledge, NIL and NiCad are among the most recent and accessible options available for our work. However, NIL often detects lower-granularity clones that exist within a portion of a function, identifying similarities in fragments rather than reporting the entire function as a clone. Since our goal is to identify reusable code at the function level, which

[2]https://docs.github.com/en/rest?apiVersion=2022-11-28

Table 1. An overview of the statistics of studied systems.

Project	#IssuesCount	#Pull Requests	#Commits	#Clone-genealogies	Size (MB)
Java projects					
RxJava	3,129	3,787	6,040	1,901	137.36
glide	4,284	874	2,891	159	93.54
netty	6,172	7,164	11,196	1,455	84.50
presto	5,629	14,592	21,369	1,523	185.98
mockito	1,535	1,523	5,946	133	47.48
pinpoint	3,924	6,178	13,169	3,146	243.48
druid	4,704	9,862	12,937	2,364	317.30
realm-java	4,550	3,213	8,977	1,983	64.39
zaproxy	4,675	3,239	9,124	2,507	191.32
grpc-java	3,173	7,186	5,933	563	78.87
PocketHub	660	630	3,512	170	11.67
shardingsphere-elasticjob	1,238	993	2,255	121	43.29
checkstyle	4,863	8,472	12,549	216	132.58
swagger-core	2,801	1,639	4,211	281	18.64
graylog2-server	6,539	9,354	22,364	262	159.42
MinecraftForge	4,218	5,205	8,603	476	113.61
maxwell	1,052	970	3,739	30	23.90
XChange	1,705	3,027	12,704	1,492	46.51
Terasology	2,039	3,054	11,924	3,190	293.45
spock	927	743	2,970	35	28.11
jabref	3,927	6,149	18,809	2,532	218.07
Rajawali	1,817	682	2,268	359	82.69
framework	9,749	2,817	19,175	6,522	476.06
gatk	4,420	3,993	4,598	380	453.60
Mekanism	6,782	855	9,183	3,877	123.17
shenyu	1,947	2,905	2,989	282	50.38
baritone	3,487	527	3,129	89	14.99
janusgraph	1,527	2,027	7,009	901	58.11
iceberg	2,451	5,626	4,503	819	41.12
light-4j	1,015	815	2,129	306	10.25
C projects					
netdata	7,416	7,711	15,752	1,086	139.52
redis	6,092	6,073	11,802	1,632	131.44
mpv	8,605	3,246	50,380	1,870	98.11
radare2	8,266	13,344	30,774	1,787	167.52
hashcat	1,901	1,872	9,363	2,983	79.61
sway	4,203	3,415	7,161	811	28.97
lvgl	2,485	1,876	8,780	1,037	186.42
SoftEtherVPN	1,007	755	1,841	1,787	529.86
libevent	890	605	4,861	623	12.36
phpredis	1,529	810	2,974	677	10.07
zfs	7,677	6,844	8,678	654	120.72
FreeRDP	3,972	4,896	17,456	7,380	62.86
i3	3,570	1,302	7,480	454	13.69
nodemcu-firmware	2,110	1,474	2,410	623	110.46
klipper	4,130	2,154	5,042	146	139.89
betaflight	5,361	7,346	18,238	2,302	383.22
WinObjC	1,828	1,120	3,544	2,406	377.53
pygame	1,902	2,048	9,017	1,743	34.17
AdAway	1,730	2,186	4,044	471	39.42
lxc	1,664	2,624	11,576	1,657	34.12
flatpak	3,036	2,387	7,265	417	40.60
nng	1,055	614	1,618	320	13.32
collectd	1,790	2,334	11,791	3,143	26.58
premake-core	920	1,080	4,036	287	31.35
cleanflight	1,734	1,385	16,589	2,295	353.78
inav	4,345	4,159	13,981	1,952	263.41
dynamorio	3,777	2,434	6,035	226	106.13
InfiniTime	709	982	2,063	330	44.39
samtools	1,120	743	2,583	128	14.60
open5gs	1,499	532	4,105	1,061	62.49

is considered the fundamental building block for code reuse [16, 25, 47, 103], analyzing clones at the function granularity enables us to capture meaningful duplications in a software system's logic. NiCad [98] supports clone detection at the level of functions and code blocks. Therefore, we employ NiCad to detect function-level clones across all commits in the main development branch of each project. Nicad has good performance among clone detectors [125, 134] due to its notable precision and recall in detecting the primary types of clones (Type 1,

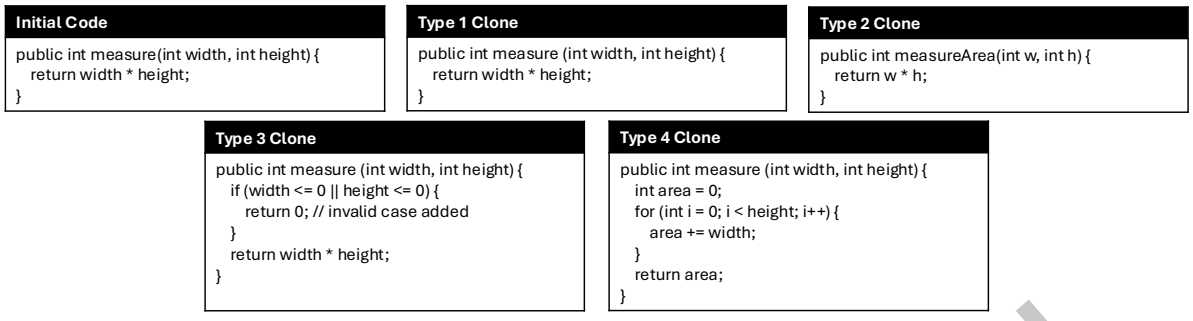

Fig. 2. Examples of Type-1 through Type-4 code clones.

Type 2, and Type 3) [96, 100, 112]. An example of Type-1 to Type-4 clones is illustrated in Figure 2. Each type is explained below:

- **Type-1 clones** refer to syntactically identical code fragments. In this example, the measure() methods implement the same logic without any modifications.
- **Type-2 clones** preserve the overall structure of the code but include minor modifications such as renaming identifiers or reformatting. For instance, the methods `measure()` and `measureArea()` both return the product of two inputs, but they differ in identifier naming (width vs. w, height vs. h) and stylistic choices. Despite these differences, the underlying logic remains the same.
- **Type-3 clones** go beyond simple renaming and may involve additions, deletions, or modifications of statements while still maintaining a high degree of similarity. In this case, both versions of `measure()` implement the same multiplication logic, but one clone introduces an additional conditional check to handle invalid input (*e.g.,* returning 0 if width or height is non-positive). Despite this added control flow, the overall structure of the code remains closely related, classifying them as Type-3 clones.
- **Type-4 clones** represent semantically equivalent code fragments that are implemented using different logic and lack strong syntactic resemblance. For example, one version of `measure()` returns the product of width and height directly, while the other computes the same result through iterative addition. Unlike Type-1 to Type-3 clones, which preserve visible textual or structural similarities, Type-4 clones require deeper semantic analysis to identify.

In this study, we restrict our focus to Type-1 through Type-3 clones, as existing tools such as NiCad, Deckard, and NIL cannot reliably detect Type-4 clones without introducing a high risk of false positives.

Moreover, Nicad supports clone detection in multiple languages, which aligns with the objectives of our study. Therefore, Nicad has been adopted in the existing work [29, 56, 79, 105, 127]. More specifically, we use the *git checkout* to retrieve the snapshot of a project for each commit and perform clone detection using the default setting in Nicad. Similar to prior work [30], we discard test files during the clone genealogy building process, since we are interested in the reuse of code in production.

2.2.2 Building Clone Group Genealogies. Clone group genealogy [57] is the evolutionary history of a clone group, i.e., the set of states and changes experienced by a clone group chronologically. During the development and maintenance stages of a project, code clones might undergo modifications. A modification to a clone can affect its size, while a change to a file containing the clone can shift the position of the clone (i.e., changes in its line numbers). Such modifications, which could introduce bugs, can be caught with each commit. Therefore, we traverse clones throughout the code snapshots by each commit in order to build the clone genealogies for each project.

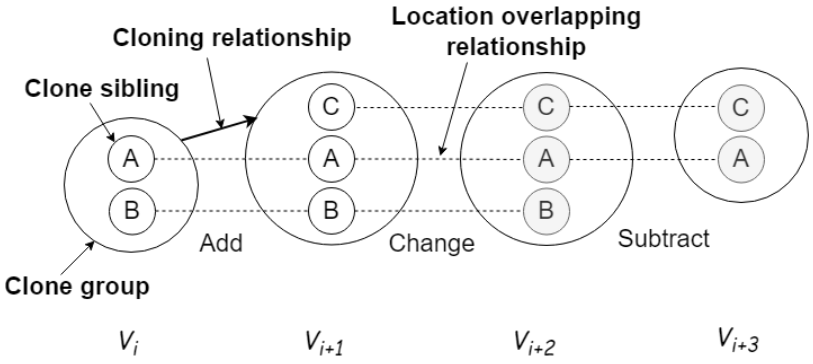

Fig. 3. Example Clone Lineage.

To build up clone group genealogies, we perform a depth-first search algorithm, starting from the inception of a clone group to its final appearance in the codebase. With the list of existing clones within a project at any specific commit generated by Nicad, we traverse through the entire commit history using the `PyDriller` [111] framework. `PyDriller` is a python-based framework that supports information extraction, such as changes in commits from each of the studied repositories. To enhance the robustness of comparing files between commits and detect all the changes to a specific file, the `Diff` tool [3] is employed in conjunction with `PyDriller` to track changes of clones to link the clone groups between each commit to assemble a chain of genealogies.

Specifically, we track the positional changes affected by the changes to code clones. The clone methods with the same fully qualified name and the same file path as well as the start line and end line number are identified and mapped between revisions to build the evolution trace. If two clone groups on the adjacent commits share clones in common, they are matched and deemed as the same clone group at different time points along the clone group genealogy. In Figure 3, for example, we see the clone group at V_{i+1} was introduced at V_i.

2.2.3 Calculating Quality Metrics for Clones. To examine the quality attributes of code clones that could be indicative of their reuse likelihood, we gather a collection of 30 software metrics as listed in Table 2. The selected metrics include product metrics (e.g., Cyclomatic Complexity, FanIn, and FanOut) at the function level, clone metrics (e.g., the length of common paths), and process metrics (e.g., the number of contributors). We utilize a static code analyzer `Understand tool` [4] to calculate source code metrics [87] for code clones, adopting its metric naming convention. These metrics include a family of complexity metrics and a diverse set of Count metrics (e.g., CountLine). While Count metrics help quantify code volume and structural attributes, the complexity metrics capture different dimensions of control-flow logic. The selected complexity metrics include:

- *Cyclomatic* measures the number of independent paths through a function's control flow.
- *CyclomaticModified* builds on *Cyclomatic* by accounting for logical operators within conditional expressions.
- *CyclomaticStrict* is a more strict estimate for cyclomatic complexity by treating compound conditions as a single decision point.
- *Sum* metrics (e.g., *SumCyclomatic*) aggregate complexity across nested functions.
- *Essential* and *Knots* metrics quantify the lack of structure and interweaving of control logic.

[3] https://git-scm.com/docs/git-diff
[4] http://www.scitools.com/

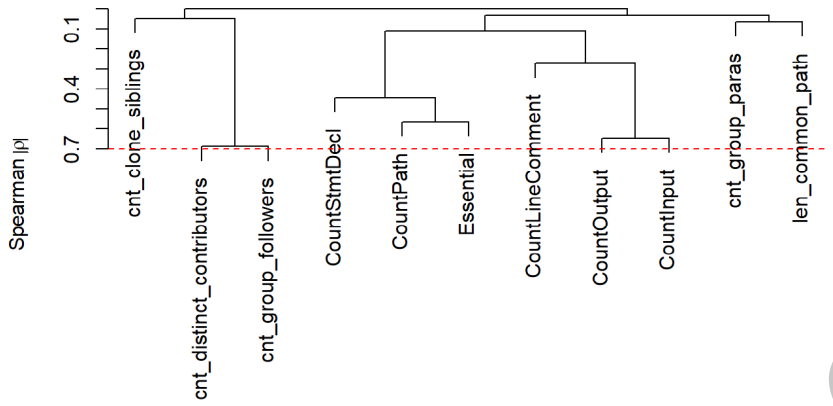

Fig. 4. Correlation analysis on the independent features to remove correlated features using the Spearman method.

Therefore, each selected metric measures a specific aspect of the code and we collected and analyzed such metrics collectively. Additionally, Python scripts are developed to extract process metrics that capture the development and evolution history of clones. In tracing the contributors associated to a clone group, we utilize `git blame`[5] to pinpoint contributors who have made modifications to the code lines in the clone group throughout its evolutionary history at a commit level.

As a code clone can have multiple siblings, we first compute the above-mentioned metrics for each sibling. Then, we aggregate the metrics to the clone group level by averaging the respective metric values of the clone siblings. The average value [119] of its clone siblings for a metric M is calculated as follow:

$$Metric(C) = \frac{\sum_{i=1}^{n} M(S_i)}{n} \tag{1}$$

where n is the number of siblings in a clone group C, and $M(S_i)$ is the value of the metric M for the clone sibling S_i in C.

2.2.4 Analyzing Feature Correlation and Redundancy. Highly correlated variables may affect the interpretation of the model and may cause spurious influences to be reported for certain studied characteristics. Therefore, prior to building ML classification models, we check the correlation and redundancy between the studied variables. We determine the correlation between studied features using Spearman's coefficients, considering pairs of features with a value of 0.7 or above as highly correlated [73, 86, 119]. For example, Figure 4 shows how *CountLineComment* and *CountLine* are highly correlated and *CountLineComment* is selected for better understanding. We also conduct the redundancy analysis before building models to avoid redundant features that could interfere with each other, the Redun function in R is used to conduct the redundancy analysis of features. No features are reported as redundant in this process [85]. The procedure for removing correlated metrics is recursively repeated on the pruned metrics set until the feature elimination algorithm reaches a fixed number of features. At this point, we enrich the final list of non-correlated and non-redundant metrics, which includes *CountFollowers, SimilarityClonePaths, CountContributors, CountLineBlank, CountInput, CountStmtDecl, CountParameters, CountLine, CountOutput, CountLineComment, Essential,* and *CountPath*, as our final metrics.

[5]https://git-scm.com/docs/git-blame

Table 2. An overview of the clone, product, and process metrics.

Metric	Metric Description	Citation
Product Metrics		
CountInput	Number of calling subprograms plus global variables read.	[119]
CountOutput	Number of called subprograms plus global variables set.	[119]
RatioCommentToCode	Ratio of comment lines to code lines.	[119]
CountPath	Number of possible paths, not counting abnormal exits or gotos.	[74]
Cyclomatic	Cyclomatic complexity.	[119]
CyclomaticModified	Modified cyclomatic complexity.	[119]
CyclomaticStrict	Strict cyclomatic complexity.	[119]
SumCyclomatic	Sum of cyclomatic complexity of all nested functions or methods.	[84]
SumCyclomaticModified	Sum of modified cyclomatic complexity of all nested functions or methods.	[75]
SumCyclomaticStrict	Sum of strict cyclomatic complexity of all nested functions or methods.	[75]
Essential	Cyclomatic complexity after replacing well-structured control structures with a single statement.	[119]
SumEssential	Sum of essential complexity of all nested functions or methods.	[75]
Knots	Measure of overlapping jumps.	[75]
MaxEssentialKnots	Maximum Knots after structured programming constructs have been removed.	[75]
MinEssentialKnots	Minimum Knots after structured programming constructs have been removed.	[75]
MaxNesting	Maximum nesting level of control constructs in the function.	[119]
CountParameters	Number of parameters passed to the function.	[91]
CountLine	Number of all physical lines.	[119]
CountLineCode	Number of lines containing source code, excluding comments.	[119]
CountLineCodeDecl	Number of lines containing declarative source code.	[119]
CountLineCodeExe	Number of lines containing executable source code.	[119]
CountSemicolon	Number of semicolons.	[75]
CountLineBlank	Number of blank lines.	[84]
CountStmt	Number of declarative plus executable statements.	[119]
CountStmtDecl	Number of declarative statements.	[119]
CountStmtExe	Number of executable statements.	[119]
CountLineComment	Number of lines containing comment.	[119]
Clone Metrics		
SimilarityClonePaths	Assessed using the Jaccard coefficient, where a lower score indicates greater diversity.	[22]
CountFollowers	Number of people following the contributors that wrote the clones.	[133]
Process Metrics		
CountContributors	Number of distinct contributors that access the clone group.	[119]

2.2.5 Building a Labeled Dataset for Reusable Code Clones. Reusable clones have less potential for introducing bugs and are therefore less likely to cause failures during future reuse. Figure 7 shows an example of the reusable and not reusable labeled clones. The reusable clone includes the *getPrivateKey* method, which retrieves a private key from a keystore. This method demonstrates higher code quality by handling errors safely—logging exceptions rather than forcefully terminating the program—thereby supporting debugging and enhancing system stability. In contrast, the not reusable clone contains the *startHadoopDruidIndexer* method, which initiates a Hadoop Druid indexer node but suffers from several issues. It invokes *system.exit()* in both normal and error flows, risking unexpected termination of the entire application. Furthermore, its exception handling relies on vague log messages and catching *Throwable*, a practice generally discouraged as it indiscriminately captures all error types. These poor coding practices increase the risk of runtime failures and make the clone fragile and difficult to reuse safely. To label code clones as reusable or not, we use the following three criteria to identify reusable code clones:

Prompt:

You are provided with a GitHub commit's details:
Commit Details: {commit_information}

Based on this information, categorize the commit as either:
- "Bug": The commit primarily addresses a problem, error, or bug in the code.
- "Other": The commit introduces non-bug-related changes such as refactoring, new features, or documentation updates.
Your output should be a Python list: ["Category", "Brief summary of the reason"]

Example Output: ["Bug", "Resolved a null pointer exception in the login module."]
STRICT RULE: Do not include any extra text in your output other than the Python list.

Fig. 5. Zero-shot prompt used with LLaMA 3 for bug-fixing commit classification

Clone prevalence shows how often a code fragment is reused. It is measured by the frequency of reuse within clone pairs and is computed by counting the number of clone siblings in a clone group. This helps assess the popularity and usefulness of a functionality in the code base.

Clone fault-resiliency measures the robustness of code clones by calculating the ratio of non-bug-fixing to bug-fixing commits, indicating how immune a clone is to faults and thus its suitability for reuse. Reusing bug-prone clones can jeopardize the quality of a project. Therefore, code clones immune to bugs are more reusable than bug-prone code clones. A bug-fixing commit serves as evidence of reported issues. Frequent bug-fixing changes (i.e., indicating the existence of bugs) present signs of inferior code duplicates that are to be sifted out, thus the number of bug-fixing commits can be used as criteria to indicate the quality of code clones. We identify bug-fixing commits by searching the links between a commit and an issue report [13]: commits referencing issue IDs (e.g., checkstyle#13411) that are mined from the GitHub issue tracking system [6], along with keywords of "bug", "defect", "fix", and "solve" [59, 68] in the commit message are extracted as bug-fixing commits.

To evaluate the effectiveness of our approach in finding the bug-fixing commits on our dataset, we manually assess 100 randomly selected commits—50 bug-related and 50 non-bug-related—from our dataset. We employ a large language model (LLM) to provide an independent and automated assessment of commit information. We utilize Llama 3:70B-Instruct [77] due to its strong performance in understanding and reasoning about code changes [24, 31]. To this end, we first use Llama 3:70B-Instruct [77] with a zero-shot prompt (as shown in Figure 5) to review each issue report and its corresponding code changes, assigning one label (*i.e.,* bug-related or other) along with the reason for assigning that label. Subsequently, a human evaluator (the second co-author, who holds a PhD in software engineering) reviews the labels generated by Llama 3, examines the reasoning behind each classification, and assigns the final labels for each commit. The issues are labeled as either *Bug* or *Other* and serve as the ground truth to evaluate our keyword-based labeling approach. The initial labels generated by Llama 3 achieve a Cohen's Kappa score of 0.82 [95] when compared to the final human-verified labels, indicating their significant agreement. We evaluate our keyword-based labeling approach against the ground truth that is verified through both human assessment and LLM output. We observe a precision of 84% and a recall of 86% from our keyword-based approach in identifying faulty commits, demonstrating its effectiveness on our dataset.

[6]https://docs.github.com/en/issues/tracking-your-work-with-issues

```
public boolean isEnableForComponent(Component invoker) {
    treeSite = getTree(invoker);
    if (treeSite != null) {
        SiteNode node = (SiteNode) treeSite.getLastSelectedPathComponent();
        if (node != null) {
            this.setEnabled(true);
        } else {
            this.setEnabled(false);
        }
        return true;
    }
    return false;
}
```

Fig. 6. An Example of Conflict in Manual Evaluation.

Clone longevity measures how long a clone group persists in the codebase, indicating its ongoing reuse possibility, and is calculated by the number of commits the clone genealogy spans in chronological order.

To create a labeled dataset for reusable clones, we collect clones by considering all three criteria, i.e., clone prevalence, clone longevity, and clone fault-resiliency. A threshold of 0.5 is used to grade clones in each criteria in a binary manner. Taking clone longevity as an example, clones that possess a genealogy length above the top 50% (i.e., above the median value) among all the clones are seen as longevous, thus could be potentially reused. Code clones are labeled as reusable only when the clones are from the top 50% by all the above three criteria. We also measure the precision of our labeling for the reusable code clones. With a confidence level of 95% and a confidence interval of 5%, we retrieve statistically significant 382 samples out of 63,624 reusable clone dataset. The first author of the paper, a Ph.D. student majoring in computer science perform the manual investigation. We use Cohen's Kappa [95] to measure the degree of agreement between the evaluators. We reached a complete agreement in the resolution phase from an initial Cohen's Kappa of 0.56. According to our manual investigation, we obtain 85% of precision in identifying reusable code clones. For example, as shown in Figure 6, one annotator classified this code as check/verify, while another labeled it as configuration. After resolving the conflict, they both agreed that it should be categorized as a check/verify case, as the method checks whether a tree node is selected, enables or disables the component accordingly, and returns a boolean value rather than establishing a configuration. We also generate different datasets by varying the thresholds of each criteria from 0.3 to 0.7, in order to understand the sensitivity of the performance of the classifiers to the thresholds of each criteria.

2.2.6 Handling Class Imbalance. The dataset used in this study has an average imbalance ratio of 2.42 as we consider code clones as reusable only when they meet all three criteria. To address the class imbalance problem, we employ the synthetic minority oversampling technique (SMOTE) algorithm [18] to balance the dataset. With SMOTE, the minority classes are oversampled (i.e., those with fewer samples) and the majority classes are undersampled (i.e., those with more samples). The class balancing routines are performed using Python's imbalanced-learn library [63].

3 Results

In this section, we present the results of our case study with respect to our research questions. For each research question, we present our (1) motivation, (2) approach, and (3) results, followed by our general conclusions.

```java
private static PrivateKey getPrivateKey(String filename, String password, String key) {
    PrivateKey privateKey = null;

    try {
        KeyStore keystore = KeyStore.getInstance("JKS");
        keystore.load(JwtHelper.class.getResourceAsStream(filename),
                password.toCharArray());

        privateKey = (PrivateKey) keystore.getKey(key,
                password.toCharArray());
    } catch (Exception e) {
        logger.error("Exception:", e);
    }

    if (privateKey == null) {
        logger.error("Failed to retrieve private key from keystore");
    }

    return privateKey;
}
```

Reusable Code Clone

```java
public void startHadoopDruidIndexer(String[] args) {
    if (args.length != 1) {
        printHelp(); // No exit code here, leading to unexpected behavior
        System.exit(2); // Forces termination without proper logging or resource cleanup
    }

    // Unsafe instantiation of node without validating arguments
    HadoopDruidIndexerNode node = HadoopDruidIndexerNode.builder().build();

    // Lifecycle is added but never checked for null or errors in setup
    Lifecycle lifecycle = new Lifecycle();
    lifecycle.addManagedInstance(node);  // Lifecycle may not properly manage the instance

    try {
      lifecycle.start();
    }
    catch (Throwable t) {
      log.info(t, "Throwable caught at startup, committing seppuku");
      // inproper management of the thread
      Thread.sleep(500);
      printHelp();
      // Forces termination without proper logging or resource cleanup
      System.exit(1);
    }
}
```

Not Reusable Code Clone

Fig. 7. Example of Reusable and Not Reusable Code Clones.

3.1 RQ1: How well can we classify the reusable code clones?

3.1.1 Motivation. Identifying reusable code clones is instrumental in enhancing code quality and establishing best practices. For newcomers, studying reusable clones offers a blueprint for developing robust, high-quality code. When the developers copy the code, it is important to know the quality of the cloned code to see if the copied code is worth being reused or not. Moreover, identifying reusable code clones helps in prioritizing refactoring initiatives. For example, gaining insight into the likelihood of code clones being reusable can inform the decision of whether converting the cloned functionality into an API is worthwhile. However, since there are a large number of clones, it is difficult for developers to manually check each clone in order to decide if the clone is reusable or not. Therefore, we wish to evaluate the performance of our proposed ML classifiers that can distinguish the reusable code clones from the non-reusable ones and provide suggestions for code reuse.

3.1.2 Approach. We build binary classifiers, in which the observed outcome for a dependent variable can have only two possible values (i.e., 1 if a clone group is reusable, and 0 otherwise), in order to determine if clones have the potential to be reused. To this end, we evaluate both ML models and LLM-based approaches, which have shown advances in popularity, scale, and adoption in recent years [89].

ML models: We use Random Forest (RF) [48], Categorical Boosting (CatBoost) [93], Extreme Gradient Boosting (XGBoost) [20], Adaptive Boosting (AdaBoost) [104], and Decision Tree (DT) (as a base classifier) for our ML models. The five classification models are all available in the scikit-learn library [90]. We use machine learning models because they are more suitable and have a better performance on structured and tabular data, whereas deep learning models, such as RNN and CNN, typically work on unstructured data, as supported by previous studies [15, 110]. We choose these models in particular as a recent literature review [55] notes, Decision Tree, Random Forest, Bayesian Network [94] are three of the most commonly used models in code clone research. While Naive Bayes assumes full attribute independence, this is often not the case in practice, especially with source code metrics. Therefore, we exclude Bayesian Network due to its stringent independence assumption. In addition, we include Gradient Boosting and Adaboost models as they are also commonly used in several software analytics studies [39, 115, 116].

Large Language Models: We utilize three popular open-source LLMs (Gemma3:12B, Llama3:8B, Mistral:7B) to evaluate their capacity to identify reusable code clones. We evaluate three prompting settings: (1) zero shot, where we provide only the candidate code and ask whether it is reusable; (2) few shot (one to five examples), where we prepend randomly sampled labeled examples of reusable clones from our dataset to illustrate their characteristics; and (3) chain of thought, where we instruct the models to reason step by step when assessing reusability. Given the runtime cost of LLM inference, we drew a simple random sample per project, sized for a 5% margin of error at 95% confidence [6, 23], yielding 16,551 clone instances evaluated per prompting setting across all projects.

Therefore, for each project, we build the classification models using each of the classifiers and also test the performance using different LLM-based approaches with different LLMs.

Evaluate Model Performance. The classification effectiveness is mainly evaluated using the Area Under the Receiver operating characteristic Curve (AUC) [119] as the AUC measure is both threshold-independent and class imbalance insensitive. AUC evaluates the overall ability of a classifier. A value close to 1 indicates a better discrimination power of the classifier, while a value of 0.5 depicts a random guess in a binary classification task. Prior studies have commonly regarded an AUC score of 0.7 as a sign of favorable performance [10, 64, 82]. Additionally, we use the F1 score and the Matthews Correlation Coefficient (MCC) [21] as supplementary evaluation metrics. The F1 score is the harmonic mean of precision and recall [33], a single metric that considers both precision and recall. MCC considers all four elements of the confusion matrix—True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN)—making it a reliable measure for evaluating classification performance under class imbalance. For a nuanced understanding of the model's ability to accurately detect positive cases while accounting for class imbalance, we use weighted precision and weighted recall. Unlike macro-averaging, which treats all classes equally, weighted averaging adjusts for class distribution by assigning importance based on the number of instances in each class, ensuring that minority class predictions (e.g., reusable clones) are fairly represented in the evaluation.

Split training and testing dataset. The dataset is split into training and testing subsets, with 80% (i.e., 50,874 samples) allocated for training. We then employ stratified 10-times 10-fold cross-validation [109, 114] in which the entire dataset is divided into 10 folds, with each fold containing the same percentage of class labels as the overall dataset. Each fold is used to test the model performance while the remaining nine folds are employed to conduct the model training, thus resulting in 10 performance measures. Such process is repeated ten times and a total of 100 performance measures are generated on each studied project.

Tune the hyperparameters of the model. To finetune our hyperparameters, grid search on top of stratified 10-times-10-fold cross-validation is applied to optimize the model performance. During the grid search, all possible parameter combinations are considered exhaustively and the parameter delivering the best model performance is selected. Suppose that a certain classifier has M parameters, and each of them has N possible values. A grid search considers a Cartesian product M * N combinations of these possible values and runs tests in each case of them. We chose this algorithm because it is adopted in software quality analysis [117, 118] and its effectiveness has been validated.

Find the optimal thresholds of the three criteria for building the training dataset. By default, we use 0.5 as the threshold to grade clones as reusable or non-reusable according to each criterion. We choose the best performing classifier to conduct the sensitivity analysis in order to find the threshold that can yield better AUC for identifying more reusable code clones correctly. We apply five different ratio thresholds (i.e., 0.3, 0.4, 0.5, 0.6 and 0.7). For each threshold, we retrain our best-performing classifier using the specified threshold value across all three criteria and evaluate its performance using the MCC, the class imbalance score, and the AUC. MCC is particularly effective for imbalanced datasets, as it accounts for all components of the confusion matrix. Our objective is to find a threshold that balances classification performance (via MCC), fairness across classes (via the imbalance score), while ensuring that the AUC remains within an acceptable range. To assess variations in AUC across multiple thresholds, we employ a two-step statistical analysis: Friedman test [35], followed by Nemenyi's post-hoc test [83]. Friedman test first ascertains if there are significant differences in AUCs among the classifiers. If differences are found, Nemenyi's post-hoc test identifies which thresholds differ significantly in pairwise comparisons. A confidence interval of 95% (i.e., $p - value = 0.05$) is used when applying the Friedman test and Nemenyi's post-hoc test.

3.1.3 Results. **Our Random Forest classifier achieves the best AUC (i.e., 0.73) in determining reusable code clones, indicating an acceptable performance of automatically identifying reusable code clones using our product, clone and process metrics**. Across all the studied projects, we observe an acceptable AUC (≥ 0.7) for 40/60 (67%) projects, reflecting a constantly good performance across a wide range of projects. Figure 8 shows the distribution of AUC estimates of our classifiers. The experimental results achieve a median AUC of 0.67, 0.73, 0.68, 0.67, and 0.68 of the 60 projects for Decision Tree, Random Forest, CatBoost, AdaBoost, and XGBoost, respectively. The AUC value of 0.6 indicates a reasonable level of effectiveness to classify reusable code clones and the AUC value of 0.7 indicates an acceptable level of effectiveness [42].

These results indicate that our classifiers can potentially be used to automatically classify reusable code clones across various software projects, Random Forest outperforms the other models, achieving the highest median AUC of 0.73 and the highest median F1 score of 0.89, as shown in Table 3. We also observe that the performance difference among the boosting techniques (i.e., XGBClassifier, CatBoostClassifier, AdaBoostClassifier) is similar, with differences within 1 percentage point. In contrast, deep learning models such as RNN and CNN achieve lower median AUCs of 0.64 and 0.60, respectively, which may be attributed to their tendency to perform better on unstructured data [15, 110]. One reason for its superior performance could be the tabular nature of our dataset, which makes tree-based classifiers like Random Forest more effective [43]. Random Forest constructs multiple diverse trees, reducing over-reliance on any single feature. Additionally, being a non-parametric method, Random-Forest does not require the data to be normally distributed, while gradient boosting techniques like CatBoost and XGBoost may falter without extensive data preparation and scaling. Our experiments mainly use numerical data, but CatBoost's specialization in categorical variables doesn't confer a significant advantage, potentially making Random Forest more robust in achieving a higher AUC. The fact that Random forest can achieve better performance is also evidenced by other clone studies [30, 80, 107].

Threshold of 0.5, when used to grade code clones in a binary manner based on the three reusability criteria, results in the best-balanced dataset, yielding a good MCC of 0.31 and an AUC score of 0.73. The

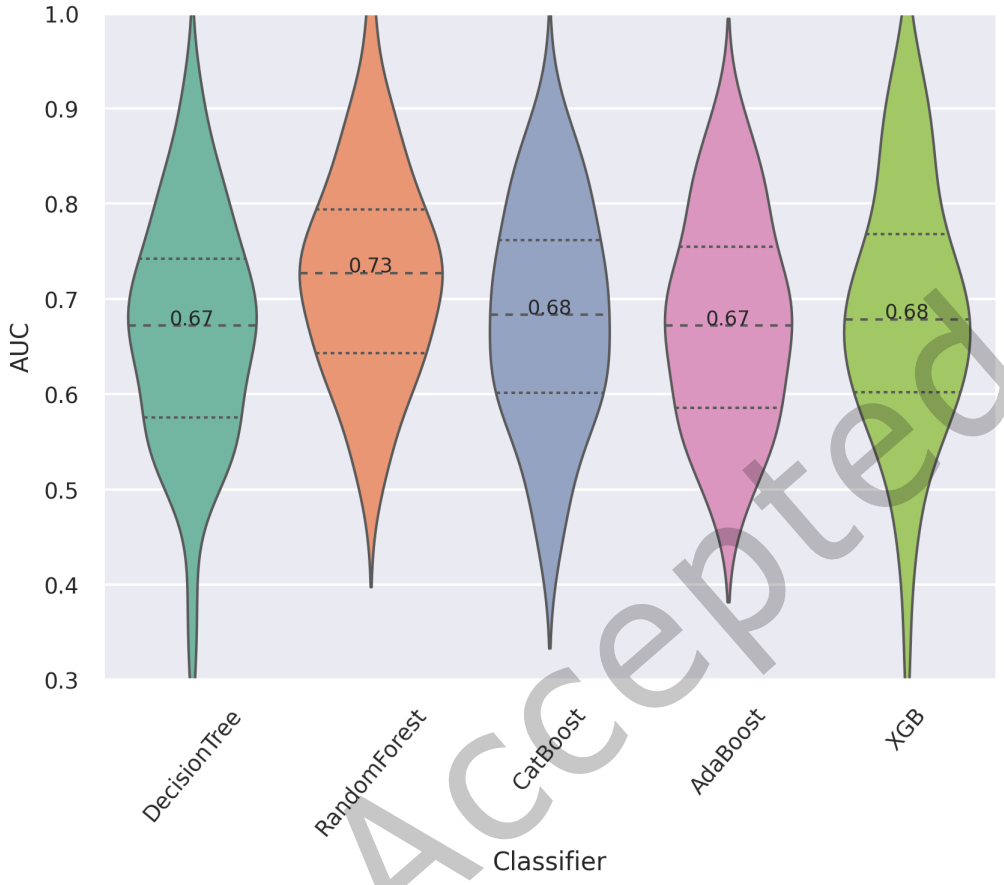

Fig. 8. Model performance in terms of AUC using five machine learning classifiers on 60 projects (i.e., 30 Java projects and 30 C projects).

Table 3. Median model performance in terms of weighted precision, recall, and F1 score

Classifier	Precision	Recall	F1
RandomForest	0.88	0.89	0.89
AdaBoost	0.85	0.81	0.83
CatBoost	0.86	0.83	0.84
DecisionTree	0.84	0.83	0.83
XGB	0.86	0.86	0.86

distributions of the AUC values of 60 projects with different thresholds show statistically significant differences when the threshold ranges from 0.3 to 0.7, confirmed with the $p - value$ of 1.31e-24 ($\ll 0.05$) by the Friedman

Table 4. Performance of different LLMs in detecting reusable code clones under varying prompting strategies.

Model Name		Zero-shot	One-shot	2-shots	3-shot	4-shots	5-shot	Chain of thoughts
gemma3	Precision	88%	88%	87%	86%	88%	85%	87%
	Recall	60%	83%	60%	34%	19%	12%	50%
	F1	69%	85%	69%	46%	24%	11%	61%
llama3	Precision	89%	89%	87%	88%	87%	88%	86%
	Recall	19%	20%	13%	15%	15%	13%	29%
	F1	25%	22%	13%	16%	15%	14%	38%
mistral	Precision	87%	87%	86%	88%	87%	87%	87%
	Recall	39%	46%	44%	45%	44%	42%	51%
	F1	51%	57%	56%	56%	55%	53%	62%

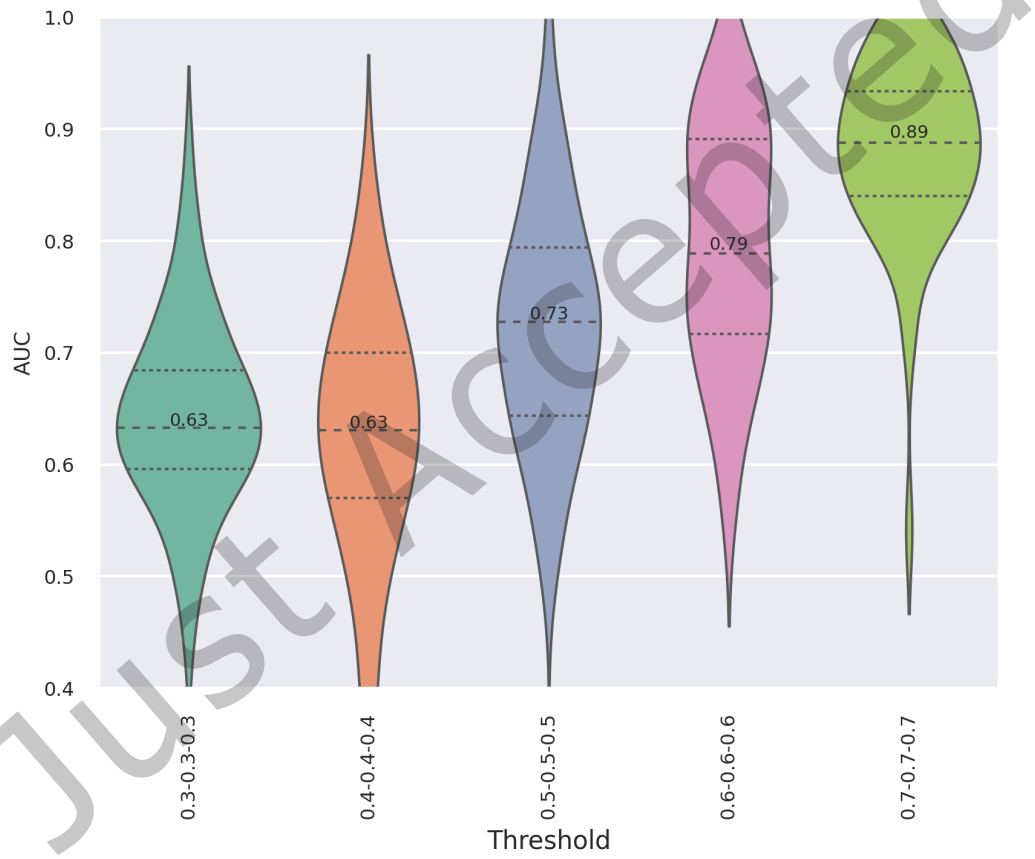

Fig. 9. Threshold sensitivity analysis according to three reusable criteria (i.e, clone prevalence, fault-resiliency, and clone prevalence) in terms of AUC using Random Forest classifier on 60 projects (i.e., 30 Java projects, 30 C projects). A threshold of 0.5 means code clones are labeled as reusable only when the clones are from the top 50% by all the above three reusable criteria.

Table 5. MCCs of the Random Forest models in different thresholds for identifying reusable code clones.

Threshold	**0.3**	**0.4**	**0.5**	**0.6**	**0.7**
MCC	0.24	0.22	0.31	0.26	0.21
Class Imbalance	20%	16%	12%	7%	3%

test, leading to the rejection of the null hypothesis that there are no differences in the AUC performances across various thresholds. Figure 9 indicates that stricter criteria (i.e., higher threshold for each criteria) for clone reusability lead to higher model performance in terms of AUC in identifying reusable code clones. Following the Friedman test, we apply Nemenyi's post-hoc test to make pairwise comparisons of AUC performance at different thresholds. We categorize thresholds into either the same or different performance groups based on their AUC similarities. Specifically, Nemenyi scores of 0.9 for thresholds 0.3 and 0.4, and 0.82 for thresholds 0.6 and 0.7, suggest that 0.3 and 0.4 belong to one performance group, while 0.6 and 0.7 belong to another. The threshold of 0.5 is distinct from both of these groups.

Similar to [11], we report the average MCC and the class imbalance percentages in Table 5 for each threshold in the reusable clone classification. The average MCCs for these models at thresholds of 0.3, 0.4, 0.5, 0.6, and 0.7 are 0.24, 0.22, 0.31, 0.26, and 0.21, respectively. Given the imbalanced nature of our reusable clone dataset, the observed MCC results are considered reasonable [7] [8]. For the threshold of 0.5, we achieve an imbalance rate of 12%, which we consider a reasonable trade-off while maximizing the MCC to 0.31. Increasing the threshold to 0.6 or 0.7 further increases the imbalance to 7% and 3%, respectively, while limiting the amount of available data. Therefore, we select the threshold of 0.5, which provides a balanced dataset with sufficient representation, while also yielding good MCC performance.

Gemma3 LLM achieves an F1 score of 85% for detecting reusable code clones, under the one-shot prompting setting. The results of our LLM approaches are shown in Table 4. Gemma3 achieves an F1 about 4% lower than Random Forest; the other LLMs and prompting variants also fall short of the ML baselines, suggesting that ML models are more effective for detecting reusable clones. One explanation is that we define reusable code clones by leveraging heuristics that human developers can readily observe, focusing on three key characteristics: clone prevalence, fault resilience, and clone longevity. LLMs receive up to a few examples of reusable clones, which are inadequate for understanding the target code and its related code history. Furthermore, providing more context, as observed with Gemma3 using more than two-shot prompting, can strain the context window and introduce effects such as recency bias [19]. As seen with Gemma3 under one-shot prompting, some models, when combined with examples, can achieve performance close to the machine learning baselines; however, LLM performance varies widely in general. Without rigorous evaluation, relying solely on LLMs remains a risky choice.

Summary for RQ1. The results of our clone classification model on 60 GitHub projects show that classifiers for a majority of the projects (i.e., 40 out of 60) are able to achieve good performance with an AUC above 0.7, among which Random Forest achieves the best AUC in determining reusable code clones out of a pool of model candidates. The 0.5 threshold offers a favorable trade-off across MCC, class imbalance, and AUC.

[7] https://www.sciencedirect.com/topics/nursing-and-health-professions/correlation-coefficient
[8] https://teachingstatistics.wordpress.com/2014/11/17/r-value-in-pearson-correlation

3.2 RQ2: Which features in the model have the most explanatory power in distinguishing reusable clones from non-reusable ones?

3.2.1 Motivation. Features play a pivotal role in building the classification models. The result of our 40/60 classification models with acceptable performance (i.e., AUC ≥ 0.7) demonstrates that our classifiers can be used to assist developers in identifying reusable code clones. Understanding the most influential features and their characteristics provides actionable insights into the nature of reusable code clones, guiding developers on what to focus on when writing or refactoring code clones. Additionally, understanding the most explanatory features can help enhance development speed and overall code quality. In this RQ, we explore the most explanatory features in the models.

3.2.2 Approach. To investigate the effect of each feature, we focus on analyzing the explanatory power of each feature. We use `permutation feature importance` [8] technique to assess the influence of each feature in our models and point out the most explanatory features in classifying reusable code clones. This technique is broadly used in software quality analysis [108, 114] since it does not rely on internal model parameters. The permutation technique can measure the importance of a feature by randomly shuffling each independent feature and observing if it affects the model's performance. A larger drop in model performance (i.e., AUC) resulting from the shuffling of a feature's values indicates a higher significance of the feature. We compute the feature importance ranks for the 40 projects that have an acceptable AUC (≥ 0.7), as [66] argues that a model needs to exhibit acceptable performance to be interpreted.

For each project with an acceptable AUC, we use the `eli5`[9] package in Python to compute the significance of each feature used by the best-performing model. The average AUC drop among the 40 projects is computed for each feature. Next, we compute the frequency of each feature shown in the projects in which an importance value is greater than the median (i.e., 0.006) of all the feature importance values.

The frequency and contribution of features are summarized in Table 6, and their distributions are illustrated in Figure 10. The Wilcoxon tests show that most differences between reusable and non-reusable clones are statistically significant ($p \ll 0.05$), except *CountInput* and *Essential*. However, the corresponding Cliff's delta [69] values indicate that the effect sizes are generally negligible, with the notable exceptions of *CountFollowers* ($\delta = 0.29$, small) and *SimilarityClonePaths* ($\delta = -0.52$, large). Taken together, this suggests that although many features exhibit statistically significant distributional differences, only a few provide practically meaningful discriminative power. Specifically, reusable clones tend to have significantly fewer followers and contributors, fewer outputs, comments, and control paths, and more blank lines and common ancestry paths. These trends suggest that reusable clones are structurally simpler and more aligned, while only certain metrics (*e.g., SimilarityClonePaths*) demonstrate a strong practical impact on distinguishing clone reusability.

To compare the distribution of significant explanatory features between reusable and non-reusable code clones, we use the Mann-Whitney-Wilcoxon test [10], as our data does not follow a normal distribution [1]. The Mann-Whitney-Wilcoxon test is an unpaired, non-parametric statistical test that assesses whether the distributions of two independent samples are the same. The null hypothesis for this test posits that it is equally likely that a randomly selected value from one sample will be less than or greater than a randomly selected value from the other sample. For each feature, we formulate the following null hypothesis:

- *H0_1:* The distributions of the selected feature in reusable and non-reusable clones are the same.

If the $p-value$ of the used Wilcoxon test on the two distributions is less than 0.05, we reject the null hypothesis and conclude that the two distributions are significantly different.

[9] https://eli5.readthedocs.io/en/latest/overview.html
[10] https://sphweb.bumc.bu.edu/otlt/mph-modules/bs/bs704_nonparametric/bs704_nonparametric4.html

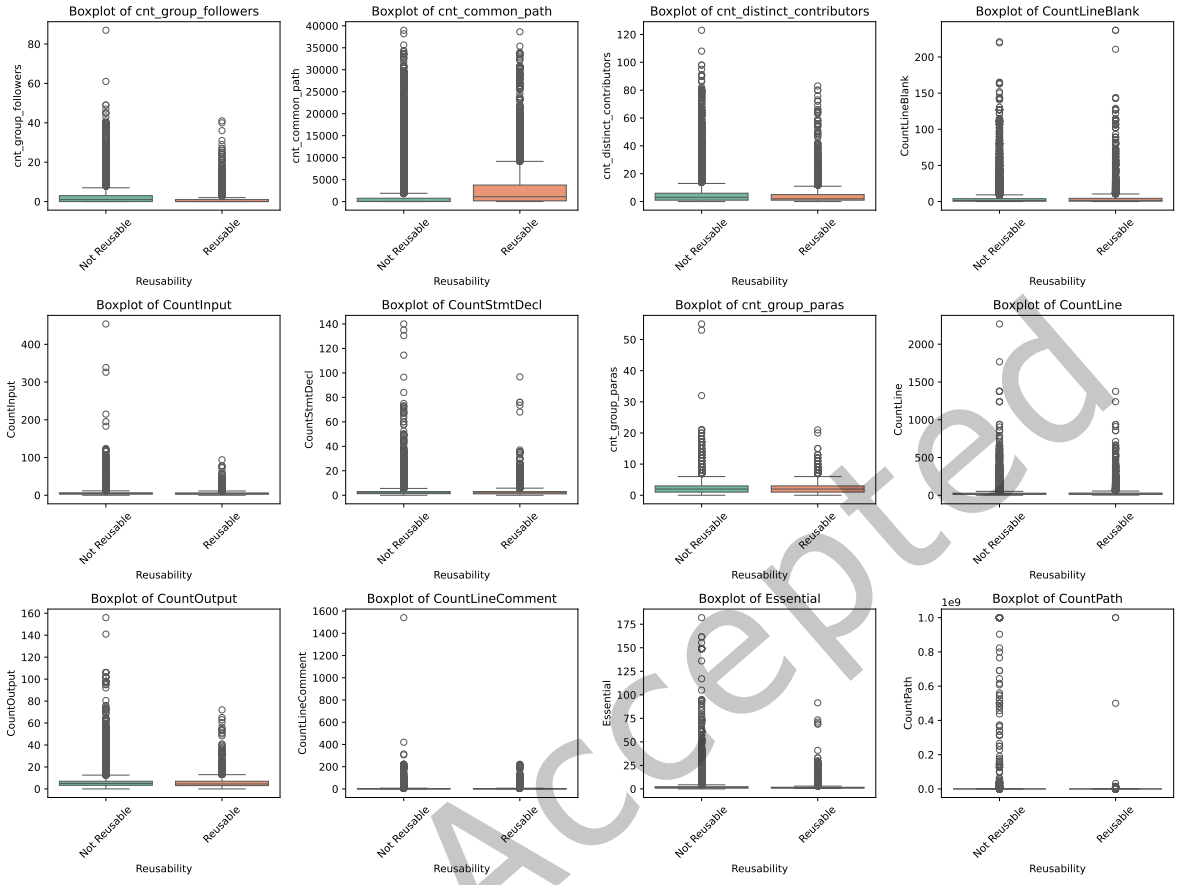

Fig. 10. The distribution of variations in the most significant features between reusable and non-reusable code clones.

Direction of the relationships. Aside from quantifying the influence of the features, we examine the direction of the relationship between the significant features (i.e., metrics) and the likelihood of a clone group being reusable. In this step, a Spearman rank correlation is used to measure the correlation between each feature and the observation, similar to [119]. In other words, a positive Spearman rank correlation shows that the feature has a positive relationship with the clone group being reusable, while a negative correlation indicates the opposite.

3.2.3 Results. **The clone metrics and process metrics are important features that affect the Random Forest model to differentiate reusable code clones from non-reusable clones.** For the 40 projects with an acceptable AUC, *CountFollowers*, *SimilarityClonePaths*, and *CountContributors* of a clone group are the top three most important features among all features used in our classifiers, with average permutation importance weights of 7.2%, 5.9%, 4.3%, respectively. In contrast, traditional complexity metrics such as *CountInput*, *CountOutput*, *Essential*, and *CountPath* demonstrate minimal influence on the model's performance, with their average permutation importance weights being markedly lower: 1%, 0.6%, 0.3%, and 0.2%, respectively. Therefore, we have the following null hypothesis:

- *H0_1: there is no difference between CountFollowers in reusable code clones and non-reusable code clones.*

Table 6. The top significant features with their contribution across projects. We report the proportion of projects where feature importance is above the median, the correlation direction (Spearman), Wilcoxon p-value for distributional difference, and Cliff's delta effect size with its magnitude.

	%AUC drop	%F1 drop	Project proportion	Spearman sign	Wilcoxon p-value	Cliff's delta	Magnitude
CountFollowers	7.25	2.79	34/40	↘	$\ll 0.05$	0.2864	small
SimilarityClonePaths	5.91	1.11	29/40	↗	$\ll 0.05$	-0.5234	large
CountContributors	4.28	2.59	24/40	↗	$\ll 0.05$	0.1722	small
CountLineBlank	1.67	0.00	19/40	↗	$\ll 0.05$	-0.0535	negligible
CountInput	0.99	1.52	12/40	↗	0.67	0.0478	negligible
CountStmtDecl	0.99	0.22	9/40	↘	0.02	0.0455	negligible
CountParameters	0.87	1.56	15/40	↗	$\ll 0.05$	-0.0477	negligible
CountLine	0.81	0.16	10/40	↗	$\ll 0.05$	-0.0068	negligible
CountOutput	0.63	1.02	9/40	↘	$\ll 0.05$	0.0610	negligible
CountLineComment	0.45	1.09	10/40	↗	$\ll 0.05$	0.0274	negligible
Essential	0.29	1.22	8/40	↗	0.60	0.0987	negligible
CountPath	0.16	0.73	9/40	↗	$\ll 0.05$	0.0683	negligible

- *H0_2: there is no difference between SimilarityClonePaths in reusable code clones and non-reusable code clones.*
- *H0_3: there is no difference between CountContributors in reusable code clones and non-reusable code clones.*

The feature *CountFollowers* is the most influential feature in identifying the likelihood of code clones being reusable in 35 out of our analyzed 40 projects. Figure 11 presents the importance of features in reusable code clones. We find a statistically significant difference ($p - value \ll 0.05$) between *CountFollowers* in reusable and non-reusable code clones. Therefore, we reject the null hypothesis *H0_1*. The Spearman rank correlation result shows that *CountFollowers* has a negative correlation with the likelihood of code clones being reusable. A further investigation into the Spearman correlation between *CountFollowers* and *CountLineComment* shows that *CountFollowers* is negatively correlated to *CountLineComment*. The more followers to the contributors that have written the clones, the fewer comments the clones likely have. In a collaborative development environment, comments is crucial for understanding the functionality and purposes of code segments. Therefore, a decrease in the number of line comments may render the code less accessible and understandable to other developers, consequently affecting the likelihood of code being reusable. Maintenance teams should prioritize integrating comprehensive comments for code clones submitted by high-profile individuals.

The feature *SimilarityClonePaths*, which indicates the diversity of clone siblings in the file system, has significant explanatory power in classifying reusable clones, suggesting that *SimilarityClonePaths* can be used as an effective metric for identifying reusable clones. Our Mann-Whitney-Wilcoxon test shows

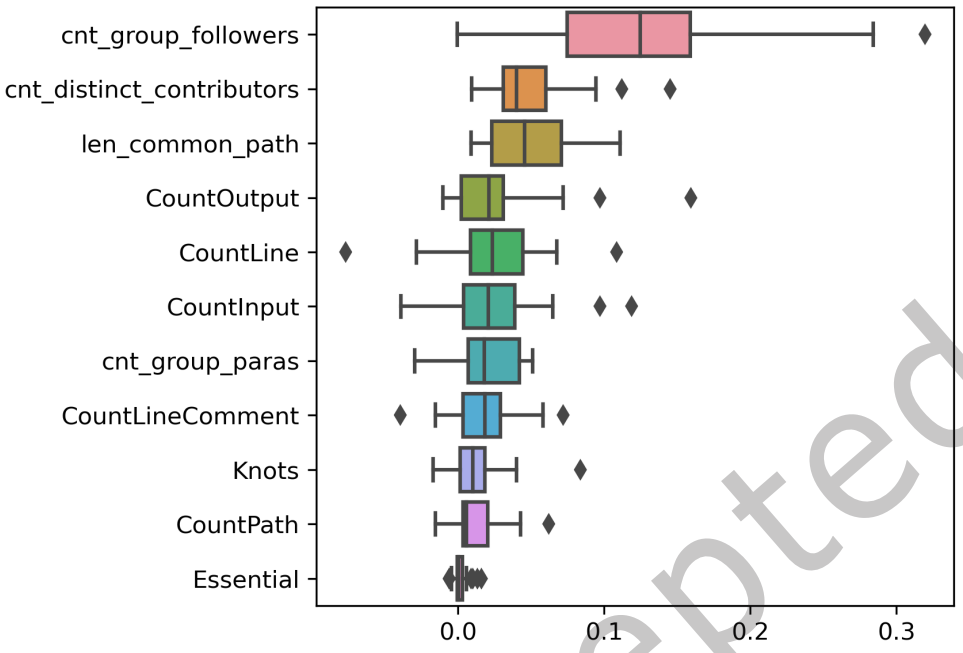

Fig. 11. Feature importance of the classifiers on 60 projects with acceptable AUC.

a $p - value$ equal to 0.03, thus *H0_2* is rejected. The statistical test reveals that the distributions of the feature *SimilarityClonePaths* between the reusable and non-reusable code clones are significantly different. The Spearman rank correlation test shows that the feature *SimilarityClonePaths* is positively correlated to the likelihood of reusable code clones, i.e., reusable code clones tend to share more similarity in clone paths within a file system. Practitioners can prioritize refactoring efforts on code clones exhibiting higher path similarities, potentially identifying them as suitable candidates for common APIs.

The feature *CountContributors* is positively correlated with the likelihood of clones being reusable, as shown by a positive Spearman rank score. We reject the null hypothesis *H0_3* based on a $p - value \ll 0.05$ and conclude that the distributions of the feature *CountContributors* between reusable and non-reusable code clones are different. The positive outcome of the Spearman correlation test shows that reusable code clones tend to have more contributors involved. According to Linus's Law [32], "Given enough eyeballs, all bugs are shallow", the involvement of more contributors can help improve the overall quality of code. This law emphasizes the significance of collective scrutiny and contribution, nurturing a sense of shared ownership and responsibility among contributors. Such collective endeavor and shared responsibility lead to continuous cycles of improvement and enhancement. This finding suggests that prioritizing code clones with a larger number of contributors for refactoring, and contemplating the extraction of code clones as a common API, could be a strategic method to promote code reuse.

Type-3 code clones are the most common form of reusable code clones. By analyzing the types of reusable clones, we found that among our reusable code clones, 7% are type-1, 8% are type-2, and 11% are type-3 clones. This suggests that clones with greater semantic differences are more likely to evolve and become reusable. As they evolve, they tend to transform into more types of clones.

Summary for RQ2. The features *CountFollowers*, *SimilarityClonePaths*, and *CountContributors* provide the most explanatory power enabling the Random Forest model to accurately identify reusable code clones. Our findings suggest that practitioners can prioritize clone refactoring efforts to code clones that share higher path similarities and involve a larger number of contributors. Additionally, it is advisable to integrate comprehensive comments for code clones submitted by high-profile individuals to facilitate code reuse.

3.3 RQ3: What are the functionalities of reusable code clones?

3.3.1 Motivation. Reusing existing code cannot be avoided given its benefits in improving developer productivity. To help identify reusable code clones, we built classifiers to identify high-quality code clones for reliable reuse in RQ1. In practice, developers often need to understand the functionality of code to reuse existing ones. Otherwise, developers would have to explicitly go through all the available reusable code when performing particular tasks. However, pinpointing such code snippets from a plethora of code clones is both tedious and time-consuming for software developers. To gain more insight into the functionality of reusable code clones, we are interested in conducting a more detailed manual analysis on the results of the classifiers and providing actionable suggestion for the practitioners. Our insight aims to assist developers in reusing code fragments with trust, to improve the usability and understandability of code clones. Therefore, in RQ3, we investigate if the clone functionality differs between reusable and non-reusable code clones to understand what types of code clones are more reusable from the developer's perspective. In addition, we investigate whether our classifiers perform better in certain types of code functionality.

3.3.2 Approach. To understand the functionality of reusable code clones, we manually label the functionality of each clone group that is classified by our classifiers, according to their semantic characteristics: method comments, inline comments, method identifiers, function context, as well as package identifiers. Since inspecting all clone groups entails a significant manual overhead, we perform a two-phase sampling, similar to the process applied in [129, 131] to obtain statistically representative clone groups from the dataset for manual analysis. The first phase (i.e., the open coding phase) entails summarizing categories of clones and code reuse functionality. The second phase (i.e., the annotation phase) involves annotating the functionality of the sampled clone groups. It is noted that for clone groups that express multiple functionalities, we tag their functionality based on the main purpose.

Summarizing categories of clones and code reuse functionality. We identify a set of functionality categories based on a subset of clone groups using open coding. To this end, we randomly sample 80 clone groups, with 20 groups each from true positives, true negatives, false positives, and false negatives. We then manually evaluate and categorize the functionalities of these clone groups—such as scanning a file, flushing to a buffer, and inserting or retrieving records from a database—which are all examples of read/write operations. The two annotators are PhD students in Computer Science, both with expertise in Java and C. One of them is the first author of this paper. The annotators independently annotate the functionality of each clone group. In cases of disagreement, they discuss their annotations to resolve conflicts and reach a consensus. Based on these finalized annotations, we derive a set of commonly reused functionality categories.

Annotating the functionality of the sampled clone groups. We randomly sample 919 clone groups out of 7,309 clone groups from the test dataset, on which 40/60 classification models have shown acceptable performance at a threshold of 0.5 for reusable characteristics. At a 95% confidence level and 5% confidence interval [130], the 919 clone groups (i.e., 188 from true positives, 364 from true negatives, 219 from false negatives, and 206 from false positives) are sampled proportionally according to the number of code clones detected from projects. The 919 clone groups are analyzed to identify functionality according to the coding developed in the previous phase. Two annotators generate new labels as needed and resolve any conflicts to generate a labeling scheme

Table 7. A taxonomy for clone group functionality.

Function Category	Category Description	Function Examples
Search	Retrieve related information to a given object	get, query, search, find, index
Config	Prepare and initialize a working environment	set, constructors, intializations
Check/Determine	Check a specific status	isValid
Add/Remove from collection	Add/insert/remove/ delete/exclude/append	append
Handle events	Listen to an event and take handling actions	onMouseMove
Convert	Convert an object from one type to another	toString
Read/Write	Read/write from/to db, file, stream, buffer	scan, flush, read, write
Math	Process calculations	sum, max, min, compare, equals
Encode/Decode	Encode or decode an object	encrypt, decrypt
Visit	Traverse a collection of objects	forEach
Cleanup	Cleanup a working environment	clean

through discussion. They then proceed to label the remaining samples. The coding process requires forty hours per annotator. We measure Cohen's Kappa with a value of 0.67, indicating a substantial level of agreement [61], to quantify the inter-rater agreement at the end of this phase. The two annotators then discuss any remaining conflicts and reach a final consensus through discussion.

3.3.3 Results. **In total, 11 functionality categories derived from the clone groups in the open coding process reveal the intention of the code, among which the top three categories are: *search, event-handling*, and *convert***. Table 7 lists the functionality taxonomy summarized from the sampled clones. Figure 12 shows the distributions of classification performance per clone functionality category. First, the number of reusable clone groups per functionality category can vary significantly. For instance, *Search* is the category with the highest number of reusable code clones while *encode/decode* and *check/verify/determine* had the least number of reusable clones, which account for less than 5% of the total. Clones in the *convert* category usually focus on transforming objects from one type to another. The *convert* category is ranked as the third highest in terms of the number of reusable code clones.

Our classifiers can distinguish reusable code clones from non-reusable ones in all the functionality categories except for the *visit* category and *encode/decode* categories. It is necessary to inspect manually the reusable code clones linked to these two categories suggested by our classifiers in RQ1. In designing and implementing functions related to *search*, *event-handling*, practitioners should consider more corner cases to ensure robustness, various use scenarios to ensure generality, and perhaps abstract them as a higher-level API and arrange them in a shallower directory for easier access if possible, since they are very likely to be reused by other developers thereafter. Besides, it is suggested to have clear code ownership during the following maintenance activities, based on our observations from RQ2.

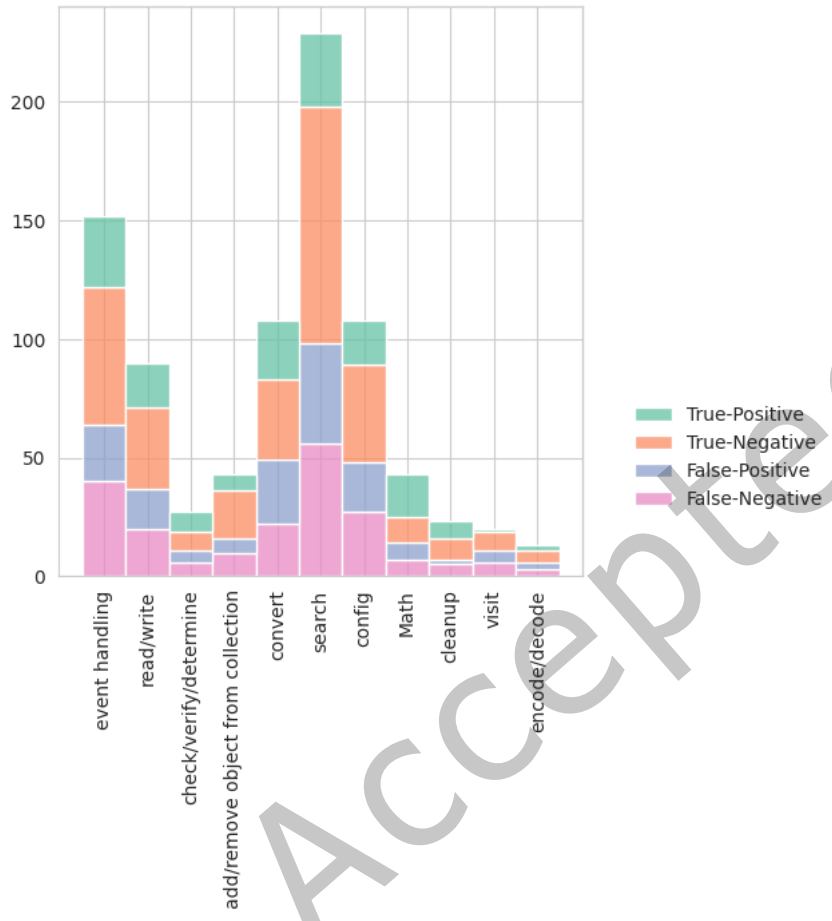

Fig. 12. Clone functionality taxonomy according to model classification of different types: True-Positive (TP), True-Negative (TN), False-Positive (FP), False-Negative (FN).

Summary for RQ3. Commonly occurred functionality categories of clones are *search* and *event-handling*. In cases where resources required to manage code clones exceed anticipated efforts, practitioners can focus on code fragments associated with *search* and *event-handling* functionality.

4 Discussion

In this section, we discuss the implications of our findings and highlight key lessons learned from our study.

4.1 Implications

We provide the implications of our findings for developers, researchers, and toolmakers.

Adoptability of Our Proposed Approach in Similar or Other Domains. In this study, we demonstrate how high-quality code clones can be classified using a set of predefined code and process metrics. Our findings and

proposed methodology can provide **practitioners** and **researchers** a direct approach for detecting high-quality code clones. This can assist practitioners in identifying reusable clones for specific problems. It also provides guidance for addressing domain-specific challenges or extending the approach across different programming languages.

Highlighting the Significance of Documentation in Reusable Code. Our findings reveal that the popularity of contributors plays a significant role in creating reusable code. We observe a significant negative correlation between highly followed contributors and the number of comments in their code, suggesting that code with fewer comments is less likely to be reusable. Therefore, we recommend that **practitioners** include comprehensive comments to maximize the potential for reuse of the code. Furthermore, we recommend that software development **organizations** encourage practitioners to adopt documentation practices, particularly among influential contributors, to enhance the reuse of high-quality code.

Defining Metrics for Detecting Reusable Code Clones. By providing a set of significant features for identifying reusable code clones, we offer **organizations** and **tool-makers** a deeper understanding of clone characteristics. Our methodology, when adopted, could enable ranking mechanisms that highlight code clones with the highest reusability potential. **Researchers** and **tool-makers** can further investigate the creation of automated models based on our findings to efficiently extract and promote high-quality code clone candidates, potentially streamlining code reuse across projects.

Providing Functionality Categories for Reusable Code Clones. Our analysis identifies search, event-handling, and conversion functions as the most common components of reusable code clones. **Practitioners** can prioritize these components to create reusable APIs within their systems. We encourage practitioners to generalize code clones in similar components of the system into an API, promoting consistency and improving the maintainability of the codebase.

Incorporating Reusable Code Clone Detection in CI/CD Pipelines. Practitioners can integrate our proposed method into their CI/CD pipelines to automatically detect reusable code clones. This would provide valuable insights that can inform refactoring decisions and drive optimization efforts. Incorporating this approach can improve the maintainability of the codebase, enabling more efficient software development practices over time.

Providing a Dataset for Developer Learning and LLM Fine-Tuning. We present a dataset containing reusable code clones, which **researchers** can use to fine-tune Large Language Models (LLMs) for clone code optimization tasks. Additionally, this dataset provides an excellent resource for **developer**s or learners to study and learn from existing high-quality code clones. Our identified reusable code clones can also be integrated into **code search repositories**, such as FACER (Feature-driven API Usage-based Code Examples Recommender) [5], further advancing the practical use of reusable code.

4.2 Lessons Learned

Threshold Trade-offs in Clone Reusability (RQ1): Our results reveal that increasing the threshold in terms of fault-resiliency, longevity, and prevalence for categorizing reusable clones improves classification performance as expected. However, this improvement comes at the cost of increased data imbalance and reduced generalizability of our approach. A threshold of 0.5 offers a more balanced approach, providing a reasonable trade-off between model performance and dataset representation.

Unexpected Prevalence of Type-3 Clones Among Reusable Clones (RQ1): Our results indicate that a larger portion (11%) of reusable clones are of Type-3, compared to 7% of Type-1 and 8% of Type-2 clones. This was against our initial assumption that reusable clones would predominantly be those with exact or very similar copies (Types 1 and 2). This suggests that semantically different clones (Type-3) offer greater long-term value for modification and reuse due to their greater adaptability.

Process Metrics Over Code Complexity (RQ2): Contrary to our initial expectations, code complexity metrics showed less impact on predicting clone reusability than our process metrics, such as the number of followers, which proved to be significantly more predictive. This highlights the importance of developer profile factors in determining code quality and reusability.

Functional Diversity of Reusable Clones (RQ3): Our manual validation of the functionality of reusable code clones indicates that reusable clones are primarily associated with general-purpose tasks rather than domain-specific logic. We also discovered that classifying functionality requires significant manual effort and a deep understanding of the domain, which future work could address. While our taxonomy provides a framework for identifying reusable behavior, some clones exhibit overlapping functionalities, making the labeling process subjective in certain instances.

5 Threats to Validity

Our results may be subject to threats of validity, including construct validity, internal validity, and external validity.

5.1 Threats to construct validity

Threats to construct validity concern the validity of our case study design and measurement. Different clone detectors may generate varying sets of clones. In this study, we use Nicad to extract code clones from the selected projects. The result cannot be guaranteed to be free of false positives (i.e., detect clones that are not actual clones). However, Nicad is advocated to be effective in detecting clones with high precision and recall [96, 100, 112]. Furthermore, the number of faulty changes, as determined by the number of commits that address these faults, may not be entirely accurate and could potentially skew our results. An alternative method for identifying bug-inducing changes is the SZZ Algorithm [14]. However, we have explored with the SZZ Algorithm and it has shown to achieve an accuracy of only 49%. Specifically, to evaluate the effectiveness of the SZZ algorithm, we manually examined the correspondence between bug-fix commits and bug-inducing commits, confirming this lower-than-expected accuracy rate. Consequently, we believe that our methodology to use the bug-fix commits for identifying faulty changes should be more effective. Our approach considers three dimensions—clone prevalence, fault-resiliency, and clone longevity—as practical heuristics for identifying reusable code clones. However, we acknowledge that these dimensions primarily reflect aspects related to maintainability and reusability as specified in ISO/IEC 25010 standard [34], and do not directly capture broader non-functional requirements (NFRs) such as scalability. Our goal is not to address all possible quality criteria, but to provide a practical and adaptable approach that can be tailored to the needs of different use cases. Detecting Type-4 clones introduces a higher risk of false positives, since their exact functional equivalence is undecidable without executing test cases, and their surface-level implementations may appear entirely different. This limitation could weaken the reliability of our dataset by reducing the accuracy of clone detection. Therefore, we focus on Type-1, Type-2, and Type-3 clones instead of Type-4 clones.

5.2 Threats to internal validity

Threats to internal validity are concerned with the validity of our assumptions. A potential threat may stem from the analysis of functionality categories in RQ3, where we only analyze a sample of clone groups, not the entire classification results. Open coding of the full dataset is beyond our resource availability, and we use a confidence level of 95% and confidence interval of 5% to generate statistically representative samples. Furthermore, we categorize the functionalities of reusable code clones through manual evaluation. Therefore, the interpretation of these functionalities could be biased based on our annotators' judgments and may not fully reflect the intent or context originally envisioned by the developers.

5.3 Threats to external validity

Threats to external validity mainly concern the generalization of our results to a broader population of software repositories. We conduct our experiments on 60 open-source C and Java projects. This decision was based on the popularity of these programming languages and their relevance to previous clone studies. While we acknowledge that our approach is applicable to other languages, our current findings focus on C and Java, and therefore, may not be generalizable to other programming languages.

6 Related Work

Code clones are duplicated or similar code fragments dispersed in a software system, generated by copy-and-paste activities. Generally, the clones are categorized into Type-1, Type-2, Type-3 and Type-4 clones according to their textual similarity, lexical similarity, syntactic similarity and semantic similarity, respectively [101]. Code cloning is a common practice in software development, especially with the availability of myriad open-source projects. The prevalence of code clones can be both advantageous and disastrous for software development: code cloning may boost productivity and improve the quality of software by reusing the code fragments that exhibit high reliability; on the other side, it can be detrimental to the software quality as defects can be propagated inadvertently, potentially contaminating a multitude of systems. Thus, it is anticipated that high-quality code will be reused. Even though it is relatively easy for senior software developers to assess the reuse potential of code clones based on their experience and expertise as compared to the junior developers, such expertise might not be readily available, making identifying potential reusable candidates one challenging task. Various prior studies [3–5, 49, 67] investigated the potential to facilitate code clone reuse. However, the prior studies provide reuse suggestions mainly based on clone prevalence and/or API-usage related to clones, reusing code clones reliably from a quality perspective remains unstudied.

Furthermore, a series of clone quality management techniques [40, 97] have been proposed in the literature, e.g., tracking of clone evolution, analysis of clone quality, enhancing and/or refactoring of clones.

6.1 Clone Evolution Analysis

Numerous tools (i.e., iClone [44], Decard [52], CCFinder [53]) are available for clone detection. Based on the clone detection result, Thongtanunam et al. [119] develop a clone evolution classification framework. Kim et al. [58] analyze clone genealogies and build a clone genealogy extractor. They observe that extensive refactorings of short-lived clones may not be worthwhile, and long-lived clones can not be easily refactored. Cai and Kim [17] analyze long-lived clones, and report that the number of developers who modified clones and the time since the last addition or removal of a clone to its group are highly correlated with the survival time of clones. From a quality perspective, our study evaluates reusable characteristics of code clones.

6.2 Clone Bug-proneness Analysis

Code clones may introduce potential bugs into an application. Islam and Zibran [50] find that software vulnerabilities detected in code clones have higher severity of security risks compared to those in non-cloned code. Barbour et al. [12] show that adding genealogy information to a bug prediction model built using product and process metrics can increase the explanatory power of models. Our work uses the findings of bug-proneness in cloned code as one of the dimensions to create the dataset of reusable code clones.

6.3 Clone Refactoring

To improve code clone quality, clone refactoring by merging the common functionality into a single method has been attempted in the literature. Balazinska et al. [9] provide an automated clone classification technique to help developers assess refactoring opportunities for clones. Meng et al. [76] implement a fully automatic clone

removal tool called RASE that can extract common code, create new types and methods, parameterize differences in expressions, and insert return objects and exit labels.

However, none of the aforementioned work targets clone reuse potential where our work examines the correlation between the software metrics of clones and the likelihood of clones being reusable. Moreover, the growing scale and complexity of code clones reported by clone detection tools pose imperative challenges [106] to clone management. Therefore, intelligent clone management using machine learning methods is a general trend to the research community.

7 Conclusion

In this paper, we propose a Machine Learning classification approach to identify reusable code clones, achieving a median AUC of 0.73 using Random Forest, which outperforms state-of-the-art large language model baselines. Our result shows that *CountFollowers*, *CountContributors*, and *SimilarityClonePaths* of clones are the three most important features deciding the likelihood of clones being reusable. This study sheds light into identifying reusable code clones on the method level. Practitioners can leverage our classifiers to determine whether a clone group is worth reusing or not in order to make efficient use of existing code and leverage the most explanatory features to better understand reusable code clones. Our study spans a broad range of open-source projects across various domains and two popular programming languages (i.e., Java and C). Once the features are extracted from the projects, the classifiers can be project-agnostic.

Future research could extend this study by analyzing a broader range of software projects, investigating reusable code clones across additional programming languages, and incorporating developer surveys or manual spot checks. Additionally, investigating reusable code clones at higher levels of granularity, such as the class level, could provide deeper insights into their potential for reuse as components and extracting APIs from the reusable code, and contribute to the exploration of more nuanced code reuse patterns. Furthermore, future work could conduct large-scale studies benefiting from more automated or developer-aware methods for understanding clone functionality.

References

[1] Ahmad Abdellatif, Mairieli Wessel, Igor Steinmacher, Marco A Gerosa, and Emad Shihab. 2022. BotHunter: An approach to detect software bots in GitHub. In *Proceedings of the 19th International Conference on Mining Software Repositories*. ACM, Pittsburgh, PA, USA, 6–17. https://doi.org/10.1145/3524842.3527959

[2] Shamsa Abid. 2019. Recommending Related Functions from API Usage-Based Function Clone Structures. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Tallinn, Estonia) *(ESEC/FSE 2019)*. Association for Computing Machinery, New York, NY, USA, 1193–1195. https://doi.org/10.1145/3338906.3342486

[3] Shamsa Abid, Hamid Abdul Basit, and Shafay Shamail. 2022. Context-Aware Code Recommendation in Intellij IDEA. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Singapore, Singapore) *(ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 1647–1651. https://doi.org/10.1145/3540250.3558937

[4] Shamsa Abid, Salman Javed, Momna Naseem, Suleman Shahid, Hamid Abdul Basit, and Yoshiki Higo. 2017. Codeease: harnessing method clone structures for reuse. In *2017 IEEE 11th International Workshop on Software Clones (IWSC)*. IEEE, Klagenfurt, Austria, 1–7. https://doi.org/10.1109/IWSC.2017.7880505

[5] Shamsa Abid, Shafay Shamail, Hamid Abdul Basit, and Sarah Nadi. 2021. FACER: An API Usage-Based Code-Example Recommender for Opportunistic Reuse. *Empirical Softw. Engg.* 26, 6 (nov 2021), 58 pages. https://doi.org/10.1007/s10664-021-10000-w

[6] Halim Ahmad and Hasnita Halim. 2017. Determining sample size for research activities: the case of organizational research. *Selangor Business Review* 2, 1 (2017), 20–34.

[7] Tarek M. Ahmed, Weiyi Shang, and Ahmed E. Hassan. 2015. An Empirical Study of the Copy and Paste Behavior during Development. In *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*. IEEE, Florence, Italy, 99–110. https://doi.org/10.1109/MSR.2015.17

[8] Andre Altmann, Laura Tolosi, Oliver Sander, and Thomas Lengauer. 2010. Permutation importance: A corrected feature importance measure. *Bioinformatics (Oxford, England)* 26 (04 2010), 1340–7. https://doi.org/10.1093/bioinformatics/btq134

[9] M. Balazinska, E. Merlo, M. Dagenais, B. Lague, and K. Kontogiannis. 2000. Advanced clone-analysis to support object-oriented system refactoring. In *Proceedings Seventh Working Conference on Reverse Engineering*. IEEE Computer Society, Brisbane, Australia, 98–107. https://doi.org/10.1109/WCRE.2000.891457

[10] Lingfeng Bao, Xin Xia, David Lo, and Gail C. Murphy. 2021. A Large Scale Study of Long-Time Contributor Prediction for GitHub Projects. *IEEE Transactions on Software Engineering* 47, 6 (2021), 1277–1298. https://doi.org/10.1109/TSE.2019.2918536

[11] Lingfeng Bao, Xin Xia, David Lo, and Gail C. Murphy. 2021. A Large Scale Study of Long-Time Contributor Prediction for GitHub Projects. *IEEE Transactions on Software Engineering* 47, 6 (2021), 1277–1298. https://doi.org/10.1109/TSE.2019.2918536

[12] Liliane Barbour, Le An, Foutse Khomh, Ying Zou, and Shaohua Wang. 2018. An investigation of the fault-proneness of clone evolutionary patterns. *Software Quality Journal* 26 (12 2018), 1–36. https://doi.org/10.1007/s11219-017-9375-5

[13] Christian Bird, Adrian Bachmann, Eirik Aune, John Duffy, Abraham Bernstein, Vladimir Filkov, and Premkumar Devanbu. 2009. Fair and Balanced? Bias in Bug-Fix Datasets. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering* (Amsterdam, The Netherlands) *(ESEC/FSE '09)*. Association for Computing Machinery, New York, NY, USA, 121–130. https://doi.org/10.1145/1595696.1595716

[14] Markus Borg, Oscar Svensson, Kristian Berg, and Daniel Hansson. 2019. SZZ Unleashed: An Open Implementation of the SZZ Algorithm - Featuring Example Usage in a Study of Just-in-Time Bug Prediction for the Jenkins Project. In *Proceedings of the 3rd ACM SIGSOFT International Workshop on Machine Learning Techniques for Software Quality Evaluation* (Tallinn, Estonia) *(MaLTeSQuE 2019)*. Association for Computing Machinery, New York, NY, USA, 7–12. https://doi.org/10.1145/3340482.3342742

[15] Vadim Borisov, Tobias Leemann, Kathrin Seßler, Johannes Haug, Martin Pawelczyk, and Gjergji Kasneci. 2022. Deep Neural Networks and Tabular Data: A Survey. *IEEE Transactions on Neural Networks and Learning Systems* 33, 9 (2022), 3538–3567. https://doi.org/10.1109/TNNLS.2022.3229161

[16] L. Breiman. 2004. Bagging predictors. *Machine Learning* 24 (2004), 123–140. https://doi.org/10.1007/BF00058655

[17] Dongxiang Cai and Miryung Kim. 2011. An Empirical Study of Long-Lived Code Clones. In *Proceedings of the 14th International Conference on Fundamental Approaches to Software Engineering (FASE 2011)*, Vol. 6603. Springer, Berlin, Heidelberg, 432–446. https://doi.org/10.1007/978-3-642-19811-3_30

[18] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic Minority over-Sampling Technique. *J. Artif. Int. Res.* 16, 1 (jun 2002), 321–357. https://doi.org/10.5555/1622407.1622416

[19] Nuo Chen, Jiqun Liu, Xiaoyu Dong, Qijiong Liu, Tetsuya Sakai, and Xiao-Ming Wu. 2024. Ai can be cognitively biased: An exploratory study on threshold priming in llm-based batch relevance assessment. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*. Association for Computing Machinery, New York, NY, USA, 54–63. https://doi.org/10.1145/3673791.3698420

[20] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2016)*. Association for Computing Machinery, New York, NY, USA, 785–794. https://doi.org/10.1145/2939672.2939785

[21] Davide Chicco and Giuseppe Jurman. 2020. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics* 21 (01 2020). https://doi.org/10.1186/s12864-019-6413-7

[22] Moataz Chouchen, Ali Ouni, Mohamed Wiem Mkaouer, Raula Kula, and Katsuro Inoue. 2021. WhoReview: A multi-objective search-based approach for code reviewers recommendation in modern code review. *Applied Soft Computing* 100 (03 2021), 106908. https://doi.org/10.1016/j.asoc.2020.106908

[23] William G. Cochran. 1977. *Sampling Techniques* (3rd ed.). John Wiley & Sons, New York, NY, USA.

[24] Jonathan Cordeiro, Shayan Noei, and Ying Zou. 2025. LLM-Driven Code Refactoring: Opportunities and Limitations. In *2025 IEEE/ACM Second IDE Workshop (IDE)*. IEEE, Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 32–36. https://doi.org/10.1109/IDE66625.2025.00011

[25] Ben De Meester, Tom Seymoens, Anastasia Dimou, and Ruben Verborgh. 2020. Implementation-independent function reuse. *Future Generation Computer Systems* 110 (2020), 946–959. https://doi.org/10.1016/j.future.2019.10.006

[26] Marcos César de Oliveira. 2017. DRACO: Discovering Refactorings That Improve Architecture Using Fine-Grained Co-Change Dependencies. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering* (Paderborn, Germany) *(ESEC/FSE 2017)*. Association for Computing Machinery, New York, NY, USA, 1018–1021. https://doi.org/10.1145/3106237.3119872

[27] Yael Dubinsky, Julia Rubin, Thorsten Berger, Slawomir Duszynski, Martin Becker, and Krzysztof Czarnecki. 2013. An Exploratory Study of Cloning in Industrial Software Product Lines. In *2013 17th European Conference on Software Maintenance and Reengineering*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 25–34. https://doi.org/10.1109/CSMR.2013.13

[28] Yael Dubinsky, Julia Rubin, Thorsten Berger, Slawomir Duszynski, Martin Becker, and Krzysztof Czarnecki. 2013. An Exploratory Study of Cloning in Industrial Software Product Lines. In *2013 17th European Conference on Software Maintenance and Reengineering*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 25–34. https://doi.org/10.1109/CSMR.2013.13

[29] Osama Ehsan, Foutse Khomh, Ying Zou, and Dong Qiu. 2023. Ranking code clones to support maintenance activities. *Empirical Software Engineering* 28 (04 2023). https://doi.org/10.1007/s10664-023-10292-0

[30] Osama Ehsan, Foutse Khomh, Ying Zou, and Dong Qiu. 2023. Ranking code clones to support maintenance activities. *Empirical Software Engineering* 28 (04 2023). https://doi.org/10.1007/s10664-023-10292-0

[31] Lishui Fan, Jiakun Liu, Zhongxin Liu, David Lo, Xin Xia, and Shanping Li. 2025. Exploring the capabilities of llms for code-change-related tasks. *ACM Transactions on Software Engineering and Methodology* 34, 6 (2025), 1–36. https://doi.org/10.1145/3709358

[32] Danilo Favato, Daniel Ishitani, Johnatan Oliveira, and Eduardo Figueiredo. 2019. Linus's Law: More Eyes Fewer Flaws in Open Source Projects. In *Proceedings of the XVIII Brazilian Symposium on Software Quality* (Fortaleza, Brazil) *(SBQS '19)*. Association for Computing Machinery, New York, NY, USA, 69–78. https://doi.org/10.1145/3364641.3364650

[33] Peter Flach and Meelis Kull. 2015. Precision-recall-gain curves: PR analysis done right. *Advances in Neural Information Processing Systems* 28 (2015), 838–846.

[34] International Organization for Standardization. 2011. ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models. https://www.iso.org/standard/35733.html. Accessed: 2025-04-20.

[35] Milton Friedman. 1937. The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance. *J. Amer. Statist. Assoc.* 32 (1937), 675–701. https://api.semanticscholar.org/CorpusID:120581754

[36] J. E. Gaffney and T. A. Durek. 1989. Software Reuse—Key to Enhanced Productivity: Some Quantitative Models. *Inf. Softw. Technol.* 31, 5 (jun 1989), 258–267. https://doi.org/10.1016/0950-5849(89)90005-0

[37] Mohammad Gharehyazie, Baishakhi Ray, and Vladimir Filkov. 2017. Some from Here, Some from There: Cross-Project Code Reuse in GitHub. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 291–301. https://doi.org/10.1109/MSR.2017.15

[38] Mohammad Gharehyazie, Baishakhi Ray, Mehdi Keshani, Masoumeh Soleimani Zavosht, Abbas Heydarnoori, and Vladimir Filkov. 2019. Cross-Project Code Clones in GitHub. *Empirical Softw. Engg.* 24, 3 (jun 2019), 1538–1573. https://doi.org/10.1007/s10664-018-9648-z

[39] Baljinder Ghotra, Shane McIntosh, and Ahmed E. Hassan. 2015. Revisiting the Impact of Classification Techniques on the Performance of Defect Prediction Models. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. Association for Computing Machinery, New York, NY, USA, 789–800. https://doi.org/10.1109/ICSE.2015.91

[40] Simon Giesecke. 2007. Generic modelling of code clones. In *Duplication, Redundancy, and Similarity in Software*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Internationales Begegnungs- und Forschungszentrum für Informatik (IBFI), Schloss Dagstuhl, Dagstuhl, Germany, 1–23.

[41] Antonios Gkortzis, Daniel Feitosa, and Diomidis Spinellis. 2019. *A Double-Edged Sword? Software Reuse and Potential Security Vulnerabilities*. Springer, Cham, Switzerland, 187–203. https://doi.org/10.1007/978-3-030-22888-0_13

[42] Sérgio Gonçalves, Paulo Cortez, and Sérgio Moro. 2020. A Deep Learning Classifier for Sentence Classification in Biomedical and Computer Science Abstracts. *Neural Comput. Appl.* 32, 11 (jun 2020), 6793–6807. https://doi.org/10.1007/s00521-019-04334-2

[43] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why do tree-based models still outperform deep learning on typical tabular data? *Advances in neural information processing systems* 35 (2022), 507–520.

[44] Nils Göde and Rainer Koschke. 2009. Incremental Clone Detection. In *2009 13th European Conference on Software Maintenance and Reengineering*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 219–228. https://doi.org/10.1109/CSMR.2009.20

[45] Muhammad Hammad, Önder Babur, Hamid Abdul Basit, and Mark van den Brand. 2020. DeepClone: Modeling Clones to Generate Code Predictions. In *Reuse in Emerging Software Engineering Practices*, Sihem Ben Sassi, Stéphane Ducasse, and Hafedh Mili (Eds.). Springer International Publishing, Cham, 135–151. https://doi.org/10.1007/978-3-030-64694-3_9

[46] Hideaki Hata, Raula Gaikovina Kula, Takashi Ishio, and Christoph Treude. 2021. Same File, Different Changes: The Potential of Meta-Maintenance on GitHub. In *Proceedings of the 43rd International Conference on Software Engineering (ICSE '21)*. IEEE Press, Madrid, Spain, 773–784. https://doi.org/10.1109/ICSE43902.2021.00076

[47] Lars Heinemann, Florian Deissenboeck, Mario Gleirscher, Benjamin Hummel, and Maximilian Irlbeck. 2011. On the Extent and Nature of Software Reuse in Open Source Java Projects, In Top Productivity through Software Reuse. *Top Productivity through Software Reuse: 12th International Conference on Software Reuse, ICSR, Pohang, South Korea* 6727, 207–222. https://doi.org/10.1007/978-3-642-21347-2_16

[48] Tin Kam Ho. 1995. Random decision forests. In *Proceedings of 3rd International Conference on Document Analysis and Recognition*, Vol. 1. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 278–282 vol.1. https://doi.org/10.1109/ICDAR.1995.598994

[49] Tomoya Ishihara, Keisuke Hotta, Yoshiki Higo, and Shinji Kusumoto. 2013. Reusing reused code. In *2013 20th Working Conference on Reverse Engineering (WCRE)*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 457–461. https://doi.org/10.1109/WCRE.2013.6671322

[50] Md. R. Islam and Minhaz F. Zibran. 2016. A Comparative Study on Vulnerabilities in Categories of Clones and Non-cloned Code. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 3. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 8–14. https://doi.org/10.1109/SANER.2016.90

[51] Lingxiao Jiang, Ghassan Misherghi, Zhendong Su, and Stephane Glondu. 2007. Deckard: Scalable and accurate tree-based detection of code clones. In *29th International Conference on Software Engineering (ICSE'07)*. IEEE, Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 96–105. https://doi.org/10.1109/ICSE.2007.30

[52] Lingxiao Jiang, Ghassan Misherghi, Zhendong Su, and Stephane Glondu. 2007. DECKARD: Scalable and Accurate Tree-Based Detection of Code Clones. In *29th International Conference on Software Engineering (ICSE'07)*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 96–105. https://doi.org/10.1109/ICSE.2007.30

[53] Toshihiro Kamiya, Shinji Kusumoto, and Katsuro Inoue. 2002. CCFinder: A Multilinguistic Token-Based Code Clone Detection System for Large Scale Source Code. *IEEE Trans. Software Eng.* 28 (2002), 654–670. https://doi.org/10.1109/TSE.2002.1019480

[54] Cory Kapser and Michael W. Godfrey. 2006. "Cloning Considered Harmful" Considered Harmful. In *2006 13th Working Conference on Reverse Engineering*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 19–28. https://doi.org/10.1109/WCRE.2006.1

[55] Manpreet Kaur and Dhavleesh Rattan. 2023. A systematic literature review on the use of machine learning in code clone research. *Computer science review* 47 (2023), 100528–. https://doi.org/10.1016/j.cosrev.2022.100528

[56] Faizan Khan, Istvan David, Daniel Varro, and Shane McIntosh. 2023. Code Cloning in Smart Contracts on the Ethereum Platform: An Extended Replication Study. *IEEE Transactions on Software Engineering* 49, 4 (2023), 2006–2019. https://doi.org/10.1109/TSE.2022.3207428

[57] Miryung Kim, Vibha Sazawal, David Notkin, and Gail Murphy. 2005. An Empirical Study of Code Clone Genealogies. In *Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Lisbon, Portugal) *(ESEC/FSE-13)*. Association for Computing Machinery, New York, NY, USA, 187–196. https://doi.org/10.1145/1081706.1081737

[58] Miryung Kim, Vibha Sazawal, David Notkin, and Gail Murphy. 2005. An Empirical Study of Code Clone Genealogies. In *Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Lisbon, Portugal) *(ESEC/FSE-13)*. Association for Computing Machinery, New York, NY, USA, 187–196. https://doi.org/10.1145/1081706.1081737

[59] Sunghun Kim, Thomas Zimmermann, Kai Pan, and E. James Jr. Whitehead. 2006. Automatic Identification of Bug-Introducing Changes. In *21st IEEE/ACM International Conference on Automated Software Engineering (ASE'06)*. IEEE Computer Society, Tokyo, Japan, 81–90. https://doi.org/10.1109/ASE.2006.23

[60] Andreas P. Koenzen, Neil A. Ernst, and Margaret-Anne D. Storey. 2020. Code Duplication and Reuse in Jupyter Notebooks. In *2020 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE Computer Society, Dunedin, New Zealand, 1–9. https://doi.org/10.1109/VL/HCC50065.2020.9127202

[61] J Richard Landis and Gary G. Koch. 1977. The measurement of observer agreement for categorical data. *Biometrics* 33 1 (1977), 159–74. https://api.semanticscholar.org/CorpusID:11077516

[62] Maggie Lei, Hao Li, Ji Li, Namrata Aundhkar, and Dae-Kyoo Kim. 2021. Deep learning application on code clone detection: A review of current knowledge. *Journal of Systems and Software* 184 (11 2021), 111141. https://doi.org/10.1016/j.jss.2021.111141

[63] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. 2017. Imbalanced-Learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. *J. Mach. Learn. Res.* 18, 1 (jan 2017), 559–563.

[64] Stefan Lessmann, Bart Baesens, Christophe Mues, and Swantje Pietsch. 2008. Benchmarking Classification Models for Software Defect Prediction: A Proposed Framework and Novel Findings. *IEEE Transactions on Software Engineering* 34, 4 (2008), 485–496. https://doi.org/10.1109/TSE.2008.35

[65] Z. Li, S. Lu, S. Myagmar, and Y. Zhou. 2006. CP-Miner: finding copy-paste and related bugs in large-scale software code. *IEEE Transactions on Software Engineering* 32, 3 (2006), 176–192. https://doi.org/10.1109/TSE.2006.28

[66] Zachary C. Lipton. 2018. The Mythos of Model Interpretability: In Machine Learning, the Concept of Interpretability is Both Important and Slippery. *Queue* 16, 3 (jun 2018), 31–57. https://doi.org/10.1145/3236386.3241340

[67] Dongrui Liu, Dongsheng Liu, Liping Zhang, Min Hou, and Chunhui Wang. 2016. The Prediction of Code Clone Quality Based on Bayesian Network. *International Journal of Software Engineering and Its Applications* 10 (04 2016), 47–56. https://doi.org/10.14257/ijseia.2016.10.4.05

[68] Shiran Liu, Zhaoqiang Guo, Yanhui Li, Chuanqi Wang, Lin Chen, Zhongbin Sun, Yuming Zhou, and Baowen Xu. 2023. Inconsistent Defect Labels: Essence, Causes, and Influence. *IEEE Transactions on Software Engineering* 49, 2 (2023), 586–610. https://doi.org/10.1109/TSE.2022.3156787

[69] Guillermo Macbeth, Eugenia Razumiejczyk, and Rubén Daniel Ledesma. 2011. Cliff's Delta Calculator: A non-parametric effect size program for two groups of observations. *Universitas Psychologica* 10, 2 (2011), 545–555. https://doi.org/10.11144/Javeriana.upsy10-2.cdcp

[70] Christian Macho, Shane McIntosh, and Martin Pinzger. 2018. Automatically repairing dependency-related build breakage. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 106–117. https://doi.org/10.1109/SANER.2018.8330201

[71] David Mandelin, Lin Xu, Rastislav Bodík, and Doug Kimelman. 2005. Jungloid Mining: Helping to Navigate the API Jungle. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation* (Chicago, IL, USA) *(PLDI '05)*. Association for Computing Machinery, New York, NY, USA, 48–61. https://doi.org/10.1145/1065010.1065018

[72] Madhuri R. Marri, Suresh Thummalapenta, and Tao Xie. 2009. Improving software quality via code searching and mining. In *2009 ICSE Workshop on Search-Driven Development-Users, Infrastructure, Tools and Evaluation*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 33–36. https://doi.org/10.1109/SUITE.2009.5070018

[73] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E. Hassan. 2015. An empirical study of the impact of modern code review practices on software quality. *Empirical Software Engineering* 21 (04 2015). https://doi.org/10.1007/s10664-015-9381-9

[74] Nádia Medeiros, Naghmeh Ivaki, Pedro Costa, and Marco Vieira. 2020. Vulnerable Code Detection Using Software Metrics and Machine Learning. *IEEE Access* 8 (2020), 219174–219198. https://doi.org/10.1109/ACCESS.2020.3041181

[75] Nádia Medeiros, Naghmeh Ivaki, Pedro Costa, and Marco Vieira. 2023. Trustworthiness models to categorize and prioritize code for security improvement. *Journal of Systems and Software* 198 (04 2023), 111621. https://doi.org/10.1016/j.jss.2023.111621

[76] Na Meng, Lisa Hua, Miryung Kim, and Kathryn S. McKinley. 2015. Does Automated Refactoring Obviate Systematic Editing?. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. Association for Computing Machinery, New York, NY, USA, 392–402. https://doi.org/10.1109/ICSE.2015.58

[77] Meta AI. 2024. LLaMA 3 70B Instruct FP16. https://ai.meta.com/llama/. Accessed: 2025-03-04.

[78] Manishankar Mondal, Banani Roy, Chanchal K Roy, and Kevin A Schneider. 2019. An empirical study on bug propagation through code cloning. *Journal of Systems and Software* 158 (2019), 110407. https://doi.org/10.1016/j.jss.2019.110407

[79] Manishankar Mondal, Banani Roy, Chanchal K. Roy, and Kevin A. Schneider. 2020. Associating Code Clones with Association Rules for Change Impact Analysis. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 93–103. https://doi.org/10.1109/SANER48275.2020.9054846

[80] Badri Mourad, Linda Badri, Oussama Hachemane, and Alexandre Ouellet. 2017. Exploring the Impact of Clone Refactoring on Test Code Size in Object-Oriented Software. In *2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE Computer Society, Cancun, Mexico, 586–592. https://doi.org/10.1109/ICMLA.2017.00098

[81] Tasuku Nakagawa, Yoshiki Higo, and Shinji Kusumoto. 2021. Nil: large-scale detection of large-variance clones. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 830–841. https://doi.org/10.1145/3468264.3468564

[82] Jaechang Nam and Sunghun Kim. 2015. CLAMI: Defect Prediction on Unlabeled Datasets (T). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 452–463. https://doi.org/10.1109/ASE.2015.56

[83] PETER B. NEMENYI. 1963. *DISTRIBUTION-FREE MULTIPLE COMPARISONS*. Ph. D. Dissertation. Princeton University. https://proxy.queensu.ca/login?url=https://www.proquest.com/dissertations-theses/distribution-free-multiple-comparisons/docview/302256074/se-2 Copyright - Database copyright ProQuest LLC; ProQuest does not claim copyright in the individual underlying works; Last updated - 2023-02-17.

[84] Chao Ni, Cong Tian, Kaiwen Yang, David Lo, Jiachi Chen, and Xiaohu Yang. 2023. Automatic Identification of Crash-inducing Smart Contracts. In *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 108–119. https://doi.org/10.1109/SANER56733.2023.00020

[85] Shayan Noei, Heng Li, Stefanos Georgiou, and Ying Zou. 2023. An empirical study of refactoring rhythms and tactics in the software development process. *IEEE Transactions on Software Engineering* 49, 12 (2023), 5103–5119. https://doi.org/10.1109/TSE.2023.3326775

[86] Shayan Noei, Heng Li, and Ying Zou. 2025. Detecting Refactoring Commits in Machine Learning Python Projects: A Machine Learning-Based Approach. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–25. https://doi.org/10.1145/3705309

[87] Shayan Noei, Heng Li, and Ying Zou. 2025. An Empirical Study on Release-Wise Refactoring Patterns. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 403–424. https://doi.org/10.1145/3715734

[88] Marcos Oliveira, Davi Freitas, Rodrigo Bonifacio, Gustavo Pinto, and David Lo. 2019. Finding Needles in a Haystack: Leveraging Co-change Dependencies to Recommend Refactorings. *Journal of Systems and Software* 158 (09 2019), 110420. https://doi.org/10.1016/j.jss.2019.110420

[89] Rajvardhan Patil and Venkat Gudivada. 2024. A review of current trends, techniques, and challenges in large language models (llms). *Applied Sciences* 14, 5 (2024), 2074. https://doi.org/10.3390/app14052074

[90] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-Learn: Machine Learning in Python. *J. Mach. Learn. Res.* 12, null (nov 2011), 2825–2830.

[91] Norman Peitek, Sven Apel, Chris Parnin, André Brechmann, and Janet Siegmund. 2021. Program Comprehension and Code Complexity Metrics: An fMRI Study. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, New York, NY, USA, 524–536. https://doi.org/10.1109/ICSE43902.2021.00056

[92] Gustavo Pinto, Igor Steinmacher, and Marco Aurélio Gerosa. 2016. More common than you think: An in-depth study of casual contributors. In *2016 IEEE 23rd international conference on software analysis, evolution, and reengineering (SANER)*, Vol. 1. IEEE, Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 112–123. https://doi.org/10.1109/SANER.2016.68

[93] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2018. CatBoost: Unbiased Boosting with Categorical Features. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (Montréal, Canada) *(NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 6639–6649.

[94] Irina Rish. 2001. An Empirical Study of the Naïve Bayes Classifier. *IJCAI 2001 Work Empir Methods Artif Intell* 3 (01 2001).

[95] Jeanine Romano, Jeffrey D. Kromrey, Jesse Coraggio, Jeff Skowronek, and Linda Devine. 2006. Exploring Methods for Evaluating Group Differences on the NSSE and Other Surveys: Are the t-Test and Cohen's d Indices the Most Appropriate Choices. In *Proceedings of the Annual Meeting of the Southern Association for Institutional Research*. Southern Association for Institutional Research, United States, 1–51.

[96] Chanchal Roy, James Cordy, and Rainer Koschke. 2009. Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Science of Computer Programming* 74 (05 2009), 470–495. https://doi.org/10.1016/j.scico.2009.02.007

[97] Chanchal Roy, Minhaz Zibran, and Rainer Koschke. 2014. The vision of software clone management: Past, present, and future (Keynote paper). In *Proceedings of the 2014 Software Evolution Week – IEEE Conference on Software Maintenance, Reengineering, and Reverse Engineering (CSMR-WCRE 2014)*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 18–33. https://doi.org/10.1109/CSMR-WCRE.2014.6747168

[98] Chanchal K. Roy and James R. Cordy. 2008. NICAD: Accurate Detection of Near-Miss Intentional Clones Using Flexible Pretty-Printing and Code Normalization. In *2008 16th IEEE International Conference on Program Comprehension*. IEEE Computer Society, Amsterdam, The Netherlands, 172–181. https://doi.org/10.1109/ICPC.2008.41

[99] Chanchal K. Roy and James R. Cordy. 2009. A Mutation/Injection-Based Automatic Framework for Evaluating Code Clone Detection Tools. In *2009 International Conference on Software Testing, Verification, and Validation Workshops*. IEEE Computer Society, Denver, CO, USA, 157–166. https://doi.org/10.1109/ICSTW.2009.18

[100] Chanchal K. Roy and James R. Cordy. 2009. A Mutation/Injection-Based Automatic Framework for Evaluating Code Clone Detection Tools. In *2009 International Conference on Software Testing, Verification, and Validation Workshops*. IEEE Computer Society, Denver, CO, USA, 157–166. https://doi.org/10.1109/ICSTW.2009.18

[101] Chanchal K. Roy, James R. Cordy, and Rainer Koschke. 2009. Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Science of Computer Programming* 74, 7 (2009), 470–495. https://doi.org/10.1016/j.scico.2009.02.007

[102] Hitesh Sajnani, Vaibhav Saini, Jeffrey Svajlenko, Chanchal K Roy, and Cristina V Lopes. 2016. Sourcerercc: Scaling code clone detection to big-code. In *Proceedings of the 38th international conference on software engineering*. ACM, Austin, TX, USA, 1157–1168. https://doi.org/10.1145/2884781.2884877

[103] Parvinder S. Sandhu, Aashima, Priyanka Kakkar, and Shilpa Sharma. 2010. A survey on Software Reusability. In *2010 International Conference on Mechanical and Electrical Technology*. Institute of Electrical and Electronics Engineers (IEEE), Singapore, 769–773. https://doi.org/10.1109/ICMET.2010.5598467

[104] Robert E. Schapire. 2013. *Explaining AdaBoost*. Springer Berlin Heidelberg, Berlin, Heidelberg, 37–52. https://doi.org/10.1007/978-3-642-41136-6_5

[105] André Schäfer, Wolfram Amme, and Thomas S. Heinze. 2021. Stubber: Compiling Source Code into Bytecode without Dependencies for Java Code Clone Detection. In *2021 IEEE 15th International Workshop on Software Clones (IWSC)*. Institute of Electrical and Electronics Engineers, Luxembourg City, Luxembourg, 29–35. https://doi.org/10.1109/IWSC53727.2021.00011

[106] Sara Shahzad, Ammara Hussain, and Shah Nazir. 2017. A clone management framework to improve code quality of FOSS projects. In *2017 International Conference on Communication, Computing and Digital Systems (C-CODE)*. Institute of Electrical and Electronics Engineers, Islamabad, Pakistan, 253–258. https://doi.org/10.1109/C-CODE.2017.7918938

[107] Abdullah M. Sheneamer. 2020. An Automatic Advisor for Refactoring Software Clones Based on Machine Learning. *IEEE Access* 8 (2020), 124978–124988. https://doi.org/10.1109/ACCESS.2020.3006178

[108] Emad Shihab, Akinori Ihara, Yasutaka Kamei, Walid M. Ibrahim, Masao Ohira, Bram Adams, A. Hassan, and Ken ichi Matsumoto. 2013. Studying re-opened bugs in open source software. *Empirical Software Engineering* 18 (2013), 1005–1042. https://doi.org/10.1007/s10664-012-9228-6

[109] Emad Shihab, Akinori Ihara, Yasutaka Kamei, Walid M. Ibrahim, Masao Ohira, Bram Adams, Ahmed E. Hassan, and Ken-ichi Matsumoto. 2010. Predicting Re-opened Bugs: A Case Study on the Eclipse Project. In *2010 17th Working Conference on Reverse Engineering*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 249–258. https://doi.org/10.1109/WCRE.2010.36

[110] Ravid Shwartz-Ziv and Amitai Armon. 2022. Tabular data: Deep learning is not all you need. *Information Fusion* 81 (2022), 84–90.

[111] Davide Spadini, Maurício Aniche, and Alberto Bacchelli. 2018. PyDriller: Python Framework for Mining Software Repositories. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Lake Buena Vista, FL, USA) *(ESEC/FSE 2018)*. Association for Computing Machinery, New York, NY, USA, 908–911. https://doi.org/10.1145/3236024.3264598

[112] Jeffrey Svajlenko and Chanchal K. Roy. 2014. Evaluating Modern Clone Detection Tools. In *2014 IEEE International Conference on Software Maintenance and Evolution*. Institute of Electrical and Electronics Engineers, Piscataway, NJ, USA, 321–330. https://doi.org/10.1109/ICSME.2014.54

[113] Jeffrey Svajlenko and Chanchal K Roy. 2015. Evaluating clone detection tools with bigclonebench. In *2015 IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, IEEE Computer Society, Bremen, Germany, 131–140. https://doi.org/10.1109/ICSM.2015.7332459

[114] Ankur Tagra, Haoxiang Zhang, Gopi Krishnan Rajbahadur, and Ahmed E. Hassan. 2022. Revisiting Reopened Bugs in Open Source Software Systems. *Empirical Softw. Engg.* 27, 4 (jul 2022), 34 pages. https://doi.org/10.1007/s10664-022-10133-6

[115] Chakkrit Tantithamthavorn, Ahmed E. Hassan, and Kenichi Matsumoto. 2020. The Impact of Class Rebalancing Techniques on the Performance and Interpretation of Defect Prediction Models. *IEEE Transactions on Software Engineering* 46, 11 (2020), 1200–1219. https://doi.org/10.1109/TSE.2018.2876537

[116] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2016. Automated Parameter Optimization of Classification Techniques for Defect Prediction Models. In *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*. Institute of Electrical and Electronics Engineers and the Association for Computing Machinery, Austin, Texas, USA, 321–332. https://doi.org/10.1145/2884781.2884857

[117] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2016. Automated Parameter Optimization of Classification Techniques for Defect Prediction Models. In *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, New York, NY, USA, 321–332. https://doi.org/10.1145/2884781.2884857

[118] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2019. The Impact of Automated Parameter Optimization on Defect Prediction Models. *IEEE Transactions on Software Engineering* 45, 7 (2019), 683–711. https://doi.org/10.1109/TSE.2018.2794977

[119] Patanamon Thongtanunam, Weiyi Shang, and A. Hassan. 2018. Will this clone be short-lived? Towards a better understanding of the characteristics of short-lived clones. *Empirical Software Engineering* 24 (2018), 937–972. https://doi.org/10.1007/s10664-018-9645-2

[120] Suresh Thummalapenta, Luigi Cerulo, L. Aversano, and Massimiliano Di Penta. 2010. An Empirical Study on the Maintenance of Source Code Clones. *Empirical Software Engineering* 15 (02 2010), 1–34. https://doi.org/10.1007/s10664-009-9108-x

[121] tiobe. 2024. index | TIOBE - The Software Quality Company. https://www.tiobe.com/tiobe-index/ [Accessed 07-04-2025].

[122] R. Tufano, E. Aghajani, and G. Bavota. 2022. Don't Reinvent the Wheel: Towards Automatic Replacement of Custom Implementations with APIs. In *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE Computer Society, Los Alamitos, CA, USA, 394–398. https://doi.org/10.1109/ICSME55016.2022.00046

[123] Brent van Bladel and Serge Demeyer. 2020. Clone Detection in Test Code: An Empirical Evaluation. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE Computer Society, London, Ontario, Canada, 492–500. https://doi.org/10.1109/SANER48275.2020.9054798

[124] Andrew Walker, Tomas Cerny, and Eungee Song. 2020. Open-Source Tools and Benchmarks for Code-Clone Detection: Past, Present, and Future Trends. *SIGAPP Appl. Comput. Rev.* 19, 4 (jan 2020), 28–39. https://doi.org/10.1145/3381307.3381310

[125] Yuekun Wang, Yuhang Ye, Yueming Wu, Weiwei Zhang, Yinxing Xue, and Yang Liu. 2023. Comparison and evaluation of clone detection techniques with different code representations. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, Institute of Electrical and Electronics Engineers and Association for Computing Machinery, Melbourne, Victoria, Australia, 332–344. https://doi.org/10.1109/ICSE48619.2023.00039

[126] Ming Wu, Pengcheng Wang, Kangqi Yin, Haoyu Cheng, Yun Xu, and Chanchal K Roy. 2020. Lvmapper: A large-variance clone detector using sequencing alignment approach. *IEEE access* 8 (2020), 27986–27997. https://doi.org/10.1109/ACCESS.2020.2971545

[127] Shuai Xie, Foutse Khomh, and Ying Zou. 2013. An Empirical Study of the Fault-Proneness of Clone Mutation and Clone Migration. In *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR '13)*. IEEE Press, San Francisco, CA, USA, 149–158. https://doi.org/10.1109/MSR.2013.6624022

[128] Chunli Yu, Shayan Noei, Haoxiang Zhang, and Ying Zou. 2026. Replication Package. https://zenodo.org/records/10892794.

[129] Haoxiang Zhang, Shaowei Wang, Tse-Hsun Chen, and Ahmed E. Hassan. 2021. Reading Answers on Stack Overflow: Not Enough! *IEEE Transactions on Software Engineering* 47, 11 (2021), 2520–2533. https://doi.org/10.1109/TSE.2019.2954319

[130] Haoxiang Zhang, Shaowei Wang, Tse-Hsun Chen, Ying Zou, and Ahmed E. Hassan. 2021. An Empirical Study of Obsolete Answers on Stack Overflow. *IEEE Transactions on Software Engineering* 47, 4 (2021), 850–862. https://doi.org/10.1109/TSE.2019.2906315

[131] Haoxiang Zhang, Shaowei Wang, Tse-Hsun (Peter) Chen, and Ahmed E. Hassan. 2021. Are Comments on Stack Overflow Well Organized for Easy Retrieval by Developers? *ACM Transactions on Software Engineering and Methodology* 30, 2, Article 22 (feb 2021), 31 pages. https://doi.org/10.1145/3434279

[132] Ximing Zhang, Huan Xu, Qiuling Yu, Shipei Zeng, Shan Dai, Haowen Yang, and Shuhan Wu. 2024. License recommendation for open source projects in the power industry. *Information and Software Technology* 167 (2024), 107391. https://doi.org/10.1016/j.infsof.2023.107391

[133] Guoliang Zhao, Daniel Costa, and Ying Zou. 2019. Improving the pull requests review process using learning-to-rank algorithms. *Empirical Software Engineering* 24 (08 2019). https://doi.org/10.1007/s10664-019-09696-8

[134] Wenqing Zhu, Norihiro Yoshida, Toshihiro Kamiya, Eunjong Choi, and Hiroaki Takada. 2022. MSCCD: grammar pluggable clone detection based on ANTLR parser generation. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension.*

Association for Computing Machinery, New York, NY, USA, 460–470. https://doi.org/10.1145/3524610.3529161